

**DEPARTMENT OF COMPUTER SCIENCE,
COMSATS UNIVERSITY ISLAMABAD
ABBOTTABAD CAMPUS
FINAL APPROVAL**

This is to certify that we have read the thesis submitted by Areeba Khan having registration number CIIT/FA22-BSE-008/ATD in the session **SP26**. It is our judgment that this thesis is of sufficient standard to warrant its acceptance by the COMSATS University Islamabad, Abbottabad Campus for the **Bachelor of Software Engineering**.

1. External Examiner _____

Dr. Hikmat Ullah Khan

Professor

University of Wah

2. Internal Examiner _____

Dr. Syed Faraz Ahmed

3. Supervisor _____

Dr. Abdul Nasir Khan

4. Head of Department _____

Dr. Zia ur Rehman

Department of Computer Science,

COMSATS UNIVERSITY ISLAMABAD,

Abbottabad Campus.

Dated: 11/08/26

**DEPARTMENT OF COMPUTER SCIENCE,
COMSATS UNIVERSITY ISLAMABAD
ABBOTTABAD CAMPUS
FINAL APPROVAL**

This is to certify that we have read the thesis submitted by Aiman Shabbir having registration number CIIT/FA22-BSE-104/ATD in the session **SP26**. It is our judgment that this thesis is of sufficient standard to warrant its acceptance by the COMSATS University Islamabad, Abbottabad Campus for the **Bachelor of Software Engineering**.

1. External Examiner _____

Dr. Hikmat Ullah Khan

Professor

University of Wah

2. Internal Examiner _____

Dr. Syed Faraz Ahmed

3. Supervisor _____

Dr. Abdul Nasir Khan

4. Head of Department _____

Dr. Zia ur Rehman

Department of Computer Science,

COMSATS UNIVERSITY ISLAMABAD,

Abbottabad Campus.

Dated: 11/08/26



COMSATS University Islamabad
Abbottabad, Pakistan

School/College Pick and Drop Management System

By

Areeba khan CIIT/FA22-BSE-008/ATD

Aiman Shabbir CIIT/FA22-BSE-104/ATD

Supervisor

Dr. Abdul Nasir Khan

Bachelor of Science in Software Engineering (2022-2026)

The candidate confirms that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.



COMSATS University Islamabad
Abbottabad, Pakistan

School/College Pick and Drop Management System

A project presented to
COMSATS Institute of Information Technology, Islamabad

In partial fulfillment
Of the requirement for the degree of

Bachelor of Science in Software Engineering (2022-2026)

By

Areeba khan CIIT/FA22-BSE-008/ATD
Aiman Shabbir CIIT/FA22-BSE-104/ATD

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Areeba khan

Aiman Shabbir

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (SE) “School/College Pick and Drop Management System” was developed by **Areeba Khan (CIIT/FA22-BSE-008)** and **Aiman Shabbir (CIIT/FA22-BSE-104)** under the supervision of “Dr. ABDUL NASIR KHAN” that in his opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Software Engineering.

Supervisor

External Examiner

Head of Department
(Department of Software Engineering)

EXECUTIVE SUMMARY

School/College Pick and Drop Management System is a web and mobile based application that will ensure a safe, reliable and efficient transportation solution for students. Safety issues arise because parents and guardians might lack the reassurance of driver verification, the ability to track vehicles as they travel, secure payment options, and good communication. What the proposed system would do is to connect the verified drivers with their parents through a single platform.

Parents can check vehicle locations via GPS, be notified when cars are coming and going, control the transport services, make online payments, and give feedback. Drivers can create routes, vehicle information, seat availability, and ride management and administrators can confirm drivers and track system activity. It's developed with Flutter, Firebase, Fast API, Google Maps API, and Stripe Payment Gateway. It also integrates with the VADER Sentiment Analyzer to automatically classify sentiment of reviews as positive, neutral, or negative. Parents, Drivers, Admins and Super Admins are also operating in a secure environment in which they have role-based access control.

The combination of transport management, live tracking, secure payments, driver verification and feedback analysis all in one single platform makes for student safety, operational efficiency, transparency and overall improved quality of service.

ACKNOWLEDGEMENT

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “Dr. Abdul Nasir Khan”. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Areeba Khan

Aiman Shabbir

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CNIC	Computerized National Identity Card
CRUD	Create, Read, Update, Delete
Firestore Auth	Firestore Authentication
GPS	Global Positioning System
GPS	Global Positioning System
GPS	Global Positioning System
GPS	Global Positioning System
OOP	Object-Oriented Programming
OTP	One-Time Password
SDD	Software Design Description
SDK	Software Development Kit
SRS	Software Requirements Specification
UC	Use Case
UI	User Interface
UID	Unique Identifier
URL	Uniform Resource Locator
UX	User Experience
VADER	Valence Aware Dictionary and sEntiment Reasoner

Table of Contents

1. Introduction	15
1.1 Brief.....	15
1.2 Relevance to Course Modules	15
1.3 Project Background	15
1.4 Literature Review	16
1.5 Analysis from Literature Review	17
1.6 Methodology and Software Lifecycle for This Project	18
1.6.1 Rationale behind Selected Methodology	18
2. Problem Definition.....	19
2.1 Problem Statement.....	19
2.2 Deliverables and Development Requirements	19
3. Requirement Analysis	21
3.1 Use case diagram	21
3.2 Detailed Use Case.....	23
3.3 Functional Requirements	47
3.4 Non-Functional Requirements.....	54
4. Design and Architecture.....	56
4.1 Architectural design.....	56
4.2 Process flow/Representation.....	57
4.3 Design Models	62
5. Implementation.....	67
5.1. Algorithm.....	67
5.2. External APIs.....	69
5.3. User Interface	71
6. Testing and Evaluation.....	77
6.1 Manual Testing	77
6.1.1 System Testing.....	77
6.1.2 Unit Testing	78
6.1.3 Functional Testing	79
6.1.4 Integration Testing	81
6.2 Automated Testing	82
6.2.1 Tools Used	82

7. Conclusion and Future Work	85
7.1. Conclusion.....	85
7.2. Future Work.....	86
8. References.....	87

LIST OF FIGURES

Figure 3.1 Use Case Diagram	22
Figure 4.1 : Architecture Diagram	56
Figure 4.2 : Activity Diagram of Super Admin	57
Figure 4.3 : Activity Diagram of Admin	58
Figure 4.4 : Activity Diagram of Driver	59
Figure 4.5: Activity Diagram of Parent.....	60
Figure 4.6 : Json Tree.....	61
Figure 4.7: Class Diagram of Pick and Drop Management System	62
Figure 4.8: Sequence Diagram of Super Admin	63
Figure 4.9: Sequence Diagram of Admin	64
Figure 4.10: Sequence Diagram of Driver	65
Figure 4.11: Sequence Diagram of Parent	66
Figure 5.1: Parent Dashboard	72
Figure 5.2: Assigned Driver.....	72
Figure 5.3: Parent Reviews	73
Figure 5.4: Parent Payments	73
Figure 5.5: Driver Dashboard	74
Figure 5.6: Assigned Children	74
Figure 5.7: Driver Earnings	74
Figure 5.8: Driver Feedback	74
Figure 5.9: Admin Dashboard.....	75
Figure 5.10: Routes and Schools	75
Figure 5.11: Reports.....	75
Figure 5.13: Super Admin Dashboard	76
Figure 5.14: Payment Management	76
Figure 5.15: Reports.....	76
Figure 5.16: Activity Logs.....	76

LIST OF TABLES

Table 1.1: Analysis of Literature Review	17
Table 3.1 Sign Up	23
Table 3.2: Login.....	24
Table 3.3: Manage Profile.....	25
Table 3.4: View Routes.....	26
Table 3.5: Add child	27
Table 3.6: View Notifications.....	28
Table 3.7: View Driver Profile	29
Table 3.8: Track Child Location.....	31
Table 3.9: Make Payment	32
Table 3.10: Submit Feedback	33
Table 3.11: Submit Verification Document.....	34
Table 3.12: Mark Pick/Drop	37
Table 3.13: View Feedback and Ratings	38
Table 3.14: Manage Routes	39
Table 3.15: Verify Driver	40
Table 3.16: Manage Users	41
Table 3.17: View Parent and Driver Payments.....	42
Table 3.18: Manage Admins.....	43
Table 3.19: Manage Payments.....	45
Table 3.20: Logout.....	46
Table 3.21: Sign up FR-1	47
Table 3.22: Login FR-2.....	47
Table 3.23: Manage Profile FR-3	47
Table 3.24: View Routes FR-4	48
Table 3.25: Add Child FR-5	48
Table 3.26: View Notifications FR-6.....	49
Table 3.27: View Driver Profile FR-7	49
Table 3.28: Track Child Location FR-8.....	49
Table 3.29: Make Payment FR-9	50
Table 3.30: Submit Feedback FR-10	50
Table 3.31: Submit Verification Document FR-11.....	50
Table 3.32: Mark Pick/Drop FR-12	51
Table 3.33: Update Availability FR-13	51
Table 3.34: View Feedback and Ratings FR-14	52
Table 3.35: View Earnings FR-15	52
Table 3.36: Manage Routes FR-16	52
Table 3.37: Verify Driver FR-17	53
Table 3.38: Manage Users FR-18	53
Table 3.39: Manage Payments FR-19.....	54
Table 3.40: Logout FR-20.....	54
Table 5.1: Details of APIs used in the project	69

Table 6.1: System Test Cases	77
Table 6.2: Parent Login Unit Test Case.....	78
Table 6.3: Driver Approval Unit Test Case	78
Table 6.4: Driver Registration Unit Test Case.....	78
Table 6.5: Admin Login Unit Test Case	79
Table 6.6: Payment Processing Unit Test Case	79
Table 6.7: Login Functional Test Case	80
Table 6.8: Parent Dashboard Functional Test Case	80
Table 6.9: Driver Dashboard Functional Test Case.....	80
Table 6.10: Ride Workflow Functional Test Case.....	81
Table 6.11: Payment Functional Test Case.....	81
Table 6.12: Review Submission Functional Test Case.....	81
Table 6.13: Integration Test Case	82
Table 6.14: Tools Used	82

1. Introduction

This chapter gives a general idea of the School/College Pick-and-Drop Management System. This is a mobile and web based app that works on its own and is meant to make students transport safer and more efficient. The system helps parents with problems like finding reliable transportation, checking drivers, and keeping track of where cars are at all times. It brings a safety factor and transparency to the traditional pick and drop service, automates certain critical processes such as registration, monitoring and notifying, and gathers feedback.

This chapter also covers the project's background, its objectives, the issues that students are facing with transport currently and now the tools and technologies used in its creation. It also describes the system's integration in the Bachelor of Computer Science (BC) curriculum and how it complies with the rules for software and mobile applications.

1.1 Brief

The School/College Pick and Drop Management System is designed to enhance the safety and efficiency of student transportation. Sign kids up, see where the vehicles are going live, receive live notifications, and provide feedback to drivers. Drivers have the ability to update their profile, routes, seat availability and notify parents of transportation events. The SuperAdmin is in charge of managing admins and keeping track of payments. The admins will be responsible for driver checks and operation.

The system isn't reliant on any school infrastructure, giving parents complete control over their child's commute. Development uses Flutter to make apps that work on multiple platforms, Firebase to store data and verify users and python libraries do to feedback analysis. Agile methodology guides development, which lets for quick changes and use of the feedback

1.2 Relevance to Course Modules

The School/College Pick and Drop Management System was developed with the aim to ease the difficulty faced by parents of locating safe, comfortable and transparent transportation for their children. Traditional pick and drop processes rely on unverifiable drivers, manual coordination and limited communication in many areas. This may make things unsafe, cause delays, and leads to people being unsure of what to do. Parents sometimes are not able to monitor the movement of vehicles, review timetables, or provide feedback, leading to increased anxiety and reduced trust in such services.

The objective of this project is to create a dedicated platform for parents to meet verified drivers and to simplify tasks such as adding drivers, managing routes, tracking vehicles in real time and sending notifications. The platform makes sure that the students' transportation is safe, organized, and clear by adding features like driver verification, live GPS tracking seat availability management, and feedback systems. This project is not dependent on the infrastructure of any school as compared to the traditional systems. It puts parents in control of their child's journey and puts drivers and administrators in control of running things smoothly. It also includes a payment module to monitor payments. This implies you can process payments without adding an actual payment gateway now. Overall, this project implement a solution to a real world problem with technology. It brings together mobile

application development, web development, database management and real time communication together in a single solution to make a complete package for today's students' mode of travel.

1.3 Project Background

The School/College Pick and Drop Management System was developed with the aim to ease the difficulty faced by parents of locating safe, comfortable and transparent transportation for their children. Traditional pick and drop processes rely on unverifiable drivers, manual coordination and limited communication in many areas. This may make things unsafe, cause delays, and leads to people being unsure of what to do. Parents sometimes are not able to monitor the movement of vehicles, review timetables, or provide feedback, leading to increased anxiety and reduced trust in such services.

The objective of this project is to create a dedicated platform for parents to meet verified drivers and to simplify tasks such as adding drivers, managing routes, tracking vehicles in real time and sending notifications. The platform makes sure that the students' transportation is safe, organized, and clear by adding features like driver verification, live GPS tracking seat availability management, and feedback systems. This project is not dependent on the infrastructure of any school as compared to the traditional systems. It puts parents in control of their child's journey and puts drivers and administrators in control of running things smoothly. It also includes a payment module to monitor payments. This implies you can process payments without adding an actual payment gateway now. Overall, this project implement a solution to a real world problem with technology. It brings together mobile application development, web development, database management and real time communication together in a single solution to make a complete package for today's students' mode of travel.

1.4 Literature Review

Due to the use of GPS, RFID, IoT, and mobile computing technologies, student transportation safety and monitoring systems have come a long way. The first studies focused on vehicle tracking, and more recent studies emphasize real-world monitoring, automation, and communication between parents, drivers, and administrators.

The first systems were based GPS and GSM technologies. Salunke et al. (2017) proposed a system called GPS-GPRS tracking system using microcontroller which enables to monitor the location of the vehicle in real-time using a centralized server. Another GPS-GSM based school bus tracking system was developed by Jisha et al. (2017) which has an Android application to enable parents to see the bus and make an educated guess about its arrival time. However, these systems are primarily designed to monitor vehicles and lack interaction capabilities and student-level authentication.

RFID based systems were installed to make it safer. Jisha et al. (2017) used a mobile app to combine RFID with GPS and GSM to keep track students' attendance and vehicles' movements. Raad et al. (2021) took this idea even further by adding an Iot-based RFID tracking system that works with mobile data networks and cloud computing to allow for real-time monitoring and reporting. These systems do make things safer and more accountable,

but they still depend on the institutional infrastructure and dedicated installations of hardware, which makes them less flexible for independent transport services.

The latest developments target intelligent transportation systems, which are connected to the Internet of Things. The tracking system for school buses proposed by Gull et al. (2021) is based on the Internet of things(Iot) and QR codes to identify students and mobile and web apps to notify and track students in real-time. Ammar et al. (2019) also developed a real-time tracking system with the help of Google Maps and SMS to provide real-time information about the student's entry and exit from the bus. Even though these systems make communication and openness better, they are still under the control of an institution.

Hybrid systems such as those presented by Badawy et al. (2016) employ several technologies, including RFID, GPS and OBD/CAN interfaces, to track both students and vehicles. While these types of systems provide full monitoring, they are complex, costly, and difficult to install in areas of limited resources.

Overall, it is evident from the literature that there has been a definite shift from basic GPS-based tracking systems to smart Iot-based platforms. The majority of the solutions are more focused on institutions, rely on hardware, and cannot accommodate independent and parent-led transport ecosystem.

1.5 Analysis from Literature Review

Several existing systems provide partial functionalities similar to our project, but none fully address private, parent-managed student transport. The table below compares these systems with our proposed solution:

Table 1.1: Analysis of Literature Review

Application / System	Weakness / Limitation	Proposed Project Solution
Salunke et al. (2017) – GPS-GPRS tracking system	Provides only basic vehicle tracking with no real-time user interaction or student-level monitoring features.	Real-time GPS tracking integrated with Firebase Realtime Database listeners for live updates and parent–driver communication.
Jisha et al. (2017) – GPS-based Android system	Limited functionality with weak authentication and no scalable backend architecture.	Secure Firebase Authentication with role-based access (Super Admin, Admin, Parent, Driver) and real-time data synchronization.
Raad et al. (2021) – IoT RFID cloud system	Highly dependent on RFID infrastructure and centralized institutional control.	Flexible cloud-based system using Firebase Realtime Database for decentralized parent–driver communication.
Gull et al. (2021) – IoT QR system	Requires continuous internet connectivity and is designed mainly for institutional environments.	Mobile-first independent system using Firebase Realtime listeners for instant data updates without complex infrastructure.

Ammar et al. (2019) – Google Maps + SMS system	SMS-based communication is slow and lacks real-time synchronization capabilities.	Real-time tracking system using Firebase Realtime Database instead of SMS for instant status updates.
--	---	---

1.6 Methodology and Software Lifecycle for This Project

The primary methodology chosen for the development of the School/College Pick-and-Drop Management System is Object-Oriented Programming (OOP). The project uses the Agile Software Development Lifecycle (SDLC) model, which stresses flexibility, iterative development, and constant feedback. This method lets the team break down complicated features like role-based dashboards driver verification, and GPS tracking into smaller, easier-to-manage parts. It is possible to develop, test, and improve each module separately, which makes sure that they work well together and reduces mistakes. This makes sure that changes in one part of the system don't affect other parts.

1.6.1 Rationale behind Selected Methodology

The decision to adopt OOP may hinge on the ability to organize features into classes such as Parent, Driver, Admin, and Super Admin. This will make the system modular, in that it can be used and reused and is easy to maintain and expand. All of this is done in classes specific to route management, real time notifications, handling feedback etc.

The Agile SDLC (ASDLC) is selected because it enables incremental delivery and prompt testing of functional components. Mobile and web apps are constantly evolving, and Agile enables us to react to feedback from the users, such as fixing bugs or making usability enhancements, in a timely manner. Short development cycles (sprints) also help us monitor progress, prioritize development work, and ensure that features such as child safety, driver verification and real-time monitoring are operational at all stages of the development process.

2. Problem Definition

The problematic statement and its solutions are given in this chapter. Also, deliverable and development requirements are explained below in this chapter

2.1 Problem Statement

In many places parents struggle to find safe and reliable ways for their children to travel to and from school and college. Pick and drop services are mostly informal and unregulated. Accountability is low and drivers are often not checked. Parents often use personal references or local arrangements, which means that safety, punctuality and trust are more likely to be an issue.

One of the other significant problems is that of the lack of real-time communication or monitoring. The child will be away from home, and parents will not know where the car is and how to inform them about the pick up/drop off events. This results in anxiety and uncertainty, especially for delays, changes of course or emergencies.

Current solutions are limited to school-owned transport systems or simply miss out on essential functionalities like live tracking or communication and centralised management. And there is no one platform that will link parents to verified drivers and even enable them to manage their accounts and feedback.

A reliable, independent system is needed to enable parents, drivers and administrators to: monitor what is occurring; ensure their safety; collaborate swiftly. The proposed School/College Pick and Drop Management System seeks to solve these issues by providing a structured platform for user verification, real-time tracking, automatic notifier, and managing feedback. This will make the transport journey smoother for pupils and parents with increased safety, transparency and convenience, whilst strengthening the capacity of drivers and administrators to better manage transport more effectively and efficiently.

2.2 Deliverables and Development Requirements

The project will deliver a fully functional mobile and web-based Pick and Drop Management System, including the Parent Mobile application.

2.2.1 Deliverables

- **Completed Pick and Drop Management System:** A fully functional mobile and web-based application for managing student transportation independently of schools.
- **Parent Mobile Application:** Allows parents to register, add children, track live vehicle location, receive pickup/drop notifications, and submit feedback.
- **Driver Mobile Application:** Enables drivers to create profiles, upload verification documents, manage routes, update availability, and send pickup/drop status notifications.
- **Admin Web Dashboard:** Provides functionalities for driver verification, parent management, route management, and monitoring system activities.

- **Super Admin Web Panel:** Allows management of Admin accounts and handling of system-level operations including payment record management .
- **Real Time Tracking Module:** It uses GPS based tracking to find the location of the vehicle in real time to the parents and admins.
- **Notification Module:** Sends real-time alerts to parents for pickup, drop-off and important updates.
- **Feedback Management Module:** Allows parents to submit feedback which is categorized using simple logic-based processing.
- **Payment Module:** Records and manages payment transactions within the system using Stipe payment gateway integration.
- **SRS (Software Requirements Specification):** Defines the system requirements and functionalities.
- **SDD (Software Design Description):** Documentation of system design, architecture and implementation.

2.2.2 Development Requirements

- **Flutter (Dart):** Used to make the Parent and Driver modules for the mobile app.
- **React.js:** To develop web based dashboards for Admin and Super Admin.
- **Firebase Authentication:** Provides a secure way to sign in and manage access.
- **Firebase Firestore:** This is the real-time NoSQL database that holds system data, users and routes.
- **Firebase Storage:** Used to store driver documents such as CNIC, license and pictures of their cars.
- **Google Maps API:** Allows for real-time location tracking and route visualization.
- **Visual Studio Code:** IDE for writing and debugging code.
- **Incremental / Agile Methodology:** Supports incremental development and ongoing enhancements based on feedback and testing.
- **FastApi(Python):** Create secure backend Apis for Stripe payment processing and AI-based sentiment analysis
- **System Testing:** Ensures that all modules such as tracking, notifications, authentication, and dashboards are working as expected.
- **User Testing:** To assess usability, functionality, and overall user experience.

3. Requirement Analysis

Requirement analysis: It specifies the functional requirements of Pick and Drop Management System. It defines the needs of the actors – parents, drivers, admins, Super admins – and how they engage with the system. It includes analysis of key functions of the system, such as user registration, child and route management, driver verification, real-time location tracking, notifications, reviews and feedback, payments and administrative control. The requirements serve as a basis for the design and implementation of the system for the intended users.

3.1 Use case diagram

The use case diagram is used to show the functional requirements of the Pick and Drop Management System and the interactions between the various actors and the system. It defines the key participants in the system such as User/Parent, Driver, Admin, and Super Admin, and shows the primary functions conducted by each participant. The User/Parent may register and log in, edit their profile, add children, view routes and driver profiles, track the location of the child, get notified, pay and provide feedback. The Driver can upload verification documents, handle pick and drop services, update driver availability, mark pick and drop activities, and view feedback, ratings and earnings. The Admin will be responsible for handling users, routes, drivers, reviews, and parent/drivers' information (including driver verification and moderating reviews). The Super Admin has more advanced administrative duties, such as administration monitoring and payment actions. The diagram also shows the relationships between use cases like route deviation detection and alert generation, while tracking the location of a child. In summary, the use case diagram offers a general overview of the system's capabilities and the roles played by the various actors..

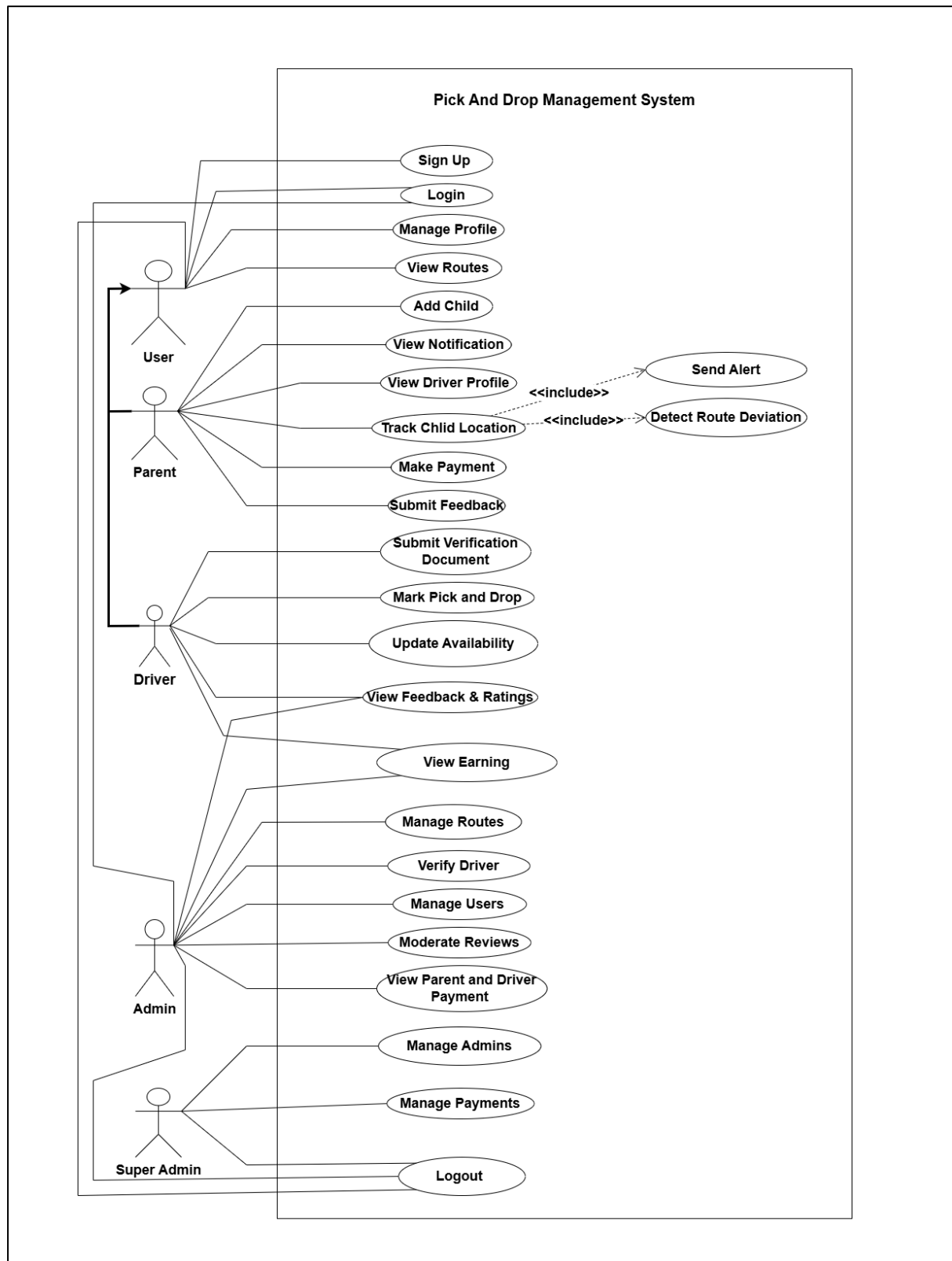


Figure 3.1: Use Case Diagram

3.2 Detailed Use Case

It describes the detailed interactions between users and the system for each major functionality.

Table 3.1 Sign Up

Use Case ID:	UC-1
Use Case Name	Sign Up
Actors	Parent / Driver
Description	A Parent or Driver logs-in to the application or website to create an account to use system services. Anyone can control the children and monitor their trips, make payments and be notified about children's trips. Drivers can schedule routes, track pick/drop, update driver availability, and upload verification documents. Ensuring a valid credential and verification by email or SMS is required for registration.
Trigger	When the user clicks the 'Sign Up' button.
Preconditions	1. The application or site loads and is operating. 2. The 'Sign Up' button is clearly and easily visible. 3. The user has an Internet connection.
Postconditions	1. A verification e-mail is sent to the user. 2. User successfully verifies to get the account active. 3. User can log in to the system and access role specific features.
Normal Flow	1. The application is opened by the user. 2. User clicks 'Sign Up' button. 3. Registration form appears with boxes for the parents/full name, email, password and role (Parent or Driver). 4. User types the necessary information. 5. The system checks the uniqueness of the e-mail. 6. The user re-enters the password. 7. The user clicks the Submit button. 8. The data of the user is saved in the system and an email with a verification is sent to the user. 9. The user clicks on the verification link. 10. The system verifies the registration is successful and the account is activated.
Alternative Flows	No alternative registration methods are supported; only standard email verification is allowed.
Exceptions	

	1.0. E2: Email has already been created. System prompts for the user to enter another e-mail. 1.0. E2: Passwords do not match. Until it receives the passwords from the system, it will not allow submission. match. 1.0. E3: The verification email/SMS message does not go to the user. System allows the user to resend. 1.0. E4: user close the form of registration. No account is made.
Business Rules	BR-1: Email must be unique for each account. BR-2: Drivers cannot access route or pick/drop features until verified.
Assumptions	1. Users know the basics of using the application. 2. User is able to read English. 3. Users can access e-mail for verification. 4. Accessibility features (screen readers, voice commands) are provided to people with disabilities.

Table 3.2: Login

Use Case ID	UC-2
Use Case Name	Login
Actors	Parent / Driver / Admin
Description	A registered user accesses the function(s) of a system by signing into the system using valid credentials. Parents can control children, record routes, make payments and be notified. Drivers have access to routes assigned to them, mark pick/drop and update availability. Administer users, view routes and payments.
Trigger	When the user clicks the 'Login' button.
Preconditions	1. The user has a registered account. 2. The application or website is loaded. 3. The user has a working internet connection.
Postconditions	1. The user is authenticated. 2. The user is redirected to their role-specific dashboard. 3. A session is created for the user.
Normal Flow	1. User launches the application or website. 2. User clicks 'Login' button. 3. The system shows the login screen (fields: email, password). 4. The user types in his/her email and password. 5. System validates credentials.

	6. If the credentials are valid, then the user is forwarded to the dashboard. 7. Record the log-in activity of the system.
Alternative Flows	Forgot password feature not implemented for this version.
Exceptions	E1: Email is not registered. Asks user to sign up. E2: Account is not verified. Asks to confirm using email. E3: Server error or system failure. System shows an error message.
Business Rules	BR-1: Users must verify their account before logging in. BR-2: Admins accounts have elevated privileges compared to Parent/Driver accounts.
Assumptions	1. Users are able to recall their logins. 2. Users have a reliable Internet connection. 3. Users can read English. 4. Users know how to navigate the applications.

Table 3.3: Manage Profile

Use Case ID	UC-3
Use Case Name	Manage Profile
Actors	Parent / Driver
Description	A registered user (Parent/Driver) can edit and access their profile information including name, contact details, email, password, profile picture and more. This enables the users to maintain the information accurate and up-to-date for system operations and communications.
Trigger	When the user clicks the 'Manage Profile' or 'Edit Profile' option in the dashboard.
Preconditions	1. User is signed in to their account. 2. System is running and connected to database(s). 3. The user is able to access the profile management interface.
Postconditions	1. The user's profile information is updated in the system. 2. Any changes will be seen on the dashboard and on the system records. 3. User receives a confirmation message.
Normal Flow	1. The user clicks on 'Manage Profile'. 2. System shows current profile information. 3. The user is able to add desired fields (like name, phone number, email, password, etc.). 4. The user gives in the corrected data. 5. The system saves the changes to the database. 6. You are notified that the profile has been updated successfully!

Alternative Flows	AF-1: User uploads a profile picture; system validates file format and size before saving.
Exceptions	1.0. E1: Input Format is invalid (email). Prompt for correct formatting. Prompt for proper formatting. 1.0. E2: Email already exists. System prevents duplication. 1.0. E3: Database connection error. System prompts for an error message. 1.0. E4: Other user cancels editing. Changes are not saved.
Business Rules	BR-1: Email and cell phone number should not be duplicated for any other user. BR-2: The requirements for password changes are at least for password security. BR-3: Drivers have to upload the required documents to finish their profile.
Assumptions	1. Users know how to navigate to profile management. 2. Users have valid credentials. 3. Users can read and understand English. 4. Users have access to the device to upload images if required.

Table 3.4: View Routes

Use Case ID	UC-4
Use Case Name	View Routes
Actors	Driver / Parent
Description	Users are able to see assigned routes (Drivers) and available routes for pick up/drop (Parents). That allows Drivers to monitor their daily journeys, and Parents to know which routes their child can take. Routes contain stops, timing and vehicle information.
Trigger	When the user selects the 'View Routes' option from the dashboard or menu.
Preconditions	1. The user is logged in. 2. Routes are already defined in the system. 3. The system has access to route and schedule data.
Postconditions	1. The requested routes are displayed to the user. 2. Users can select or note route details. 3. System logs route access activity.
Normal Flow	1. The user clicks on 'View Routes' section. 2. System provides a list of the routes that are applicable to the user role: a. Driver: Handed out routes for the day.

	b. Parent: Routes by which children will be picked up/dropped off. 3. User clicks on a route, and detailed stops, timings and vehicle/driver information appear. 4. System displays route map and relevant information.
Alternative Flows	AF-1: Parent searches routes by location, timing, or driver. AF-2: Driver filters routes by date, vehicle, or status.
Exceptions	1.0. E1: No routes were assigned or available. System reports that “No routes found.” 1.0. E2: Database or server error. Error message is present in the system. 1.0. E3: Error with filters or search criteria. Asks user to re-enter data.
Business Rules	BR-1: Drivers are only able to see routes assigned to them. BR-2: Parents are only able to see active routes for their children. BR-3: Route information should be correct and up to date, everyday.
Assumptions	1. Users are familiar with the basic navigation in the application. 2. Users are aware of the child's / vehicle's pertinent information. 3. System is connected to a working database and network. 4. English is understandable to the Users.

Table 3.5: Add child

Use Case ID	UC-5
Use Case Name	Add Child
Actors	Parent
Description	A Parent is able to set up one or more Children in their account in the system. This allows the system to provide pick up/drop off routes, notifications and payments for each child. Child details include name, age, school, class, pickup/drop addresses, contact information and profile picture.
Trigger	When the Parent selects the ‘Add Child’ option in their dashboard.
Preconditions	1. The Parent is logged into their account. 2. The system is running and accessible. 3. Parent account is verified.
Postconditions	1. Details of children are stored in the system. 2. The child is linked to the Parent account.

	3. Notifications and route assignments can be generated for the child.
Normal Flow	1. The Parent clicks on 'Add Child'. 2. A form is shown to input child information (name, age, school, class, addresses, emergency contact, profile picture). 3. The Parent completes missing information. 4. Checks for input for validity (age must be a number, addresses must be complete). 5. Parent submits the form. 6. The system stores the child's information and verifies that the child has been added. 7. The child now appears in the Parent's dashboard list.
Alternative Flows	AF-1: Parent adds multiple children at once by using a multi-entry form. AF-2: Parent imports child details from a CSV or previous records.
Exceptions	1.0. E1: All the required fields are blank. System prompts Parent to complete them. 1.0. E2: Data format is invalid (e.g., not numeric). System prompts to correct. 1.0. E3: Database error. System displays an error message. 1.0. E4: Parent cancels the operation. Child is not added.
Business Rules	BR-1: Each child must be uniquely linked to a Parent account. BR-2: Contact information is mandatory. BR-3: Child's pickup/drop address must match serviceable locations.
Assumptions	1. Parent has the child's proper personal and school information. 2. Parent is familiar with the process of completing forms. 3. The device that Parent has has internet access. 4. Users are able to read English.

Table 3.6: View Notifications

Use Case ID	UC-6
Use Case Name	View Notifications
Actors	Parent
Description	A Parent is able to see system-generated notifications regarding their children's pick-up/drop off schedule, route changes, payment,

	announcements, or alerts. This will enable Parents to be informed and act timely if necessary.
Trigger	When the Parent selects the ‘Notifications’ option in their dashboard.
Preconditions	1. Parent is logged in. 2. The system has sent alerts to the Parent or their children. 3. Device has Internet connection.
Post conditions	1. The Parent will be alerted with notifications. 2. System tracks access to notifications. 3. Any action links in notifications (e.g., payment, route updates) can be triggered.
Normal Flow	1. The Parent clicks on ‘Notifications’ section. 2. They get notifications that pertain to the Parent account. 3. Notification is displayed from newest to oldest. 4. Parent reads notifications and clicks on any actionable links, if any. 5. The system records that which notifications have been read.
Alternative Flows	AF-1: Parent filters notifications by type (e.g., payment, route, alert). AF-2: Parent marks notifications as read/unread.
Exceptions	1.0. E1: There are no notifications. You do not see any notifications in the system. 1.0. E2: Database error. An error message is displayed when using the system. 1.0. E3: Lost Internet Connection. Tells the user to reconnect.
Business Rules	BR-1: Notifications should be sent in real-time or near real-time. BR-2: Each Parent is able to see notifications of their own linked children only. BR-3: It is important to have precise timestamps and relevant information in notifications.
Assumptions	1. Parent checks notifications regularly. 2. Device supports notifications (mobile). 3. Parent can read English. 4. System is capable of generating and storing notifications.

Table 3.7: View Driver Profile

Use Case ID	UC-7
Use Case Name	View Driver Profile

Actors	Parent
Description	A Parent can see this assigned driver's profile for their child's route. Profile contains the driver's name, contact information, ratings, cars, and verification status. This enables Parents to keep a safe distance and to ask the driver for information should it be necessary.
Trigger	When the Parent selects the 'View Driver Profile' option from the child's route details or dashboard.
Preconditions	1. Parent has signed in. 2. Driver gets the Parent's child's route. 3. The driver profile data is present in the system.
Post conditions	1. The Parent is shown Driver profile details. 2. The access of the profile is logged in the system. 3. If parent provides driver with communication details, they can initiate communication with driver.
Normal Flow	1. The Parent is taken to the child's route details. 2. The system offers a choice of viewing the profile of the driver assigned to that system. 3. Parent selects 'View Driver Profile'. 4. Driver information, such as name, contact details, vehicle information, ratings, and verification status are retrieved and presented. 5. If necessary, the Parent can record information or reach out to the driver.
Alternative Flows	AF-1: Parent views multiple drivers if assigned temporarily. AF-2: Parent accesses driver ratings and feedback to assess driver performance.
Exceptions	1.0. E1: No information about the driver. Error message: "Profile not found." 1.0. E2: Database error. Error message in system. 1.0. E3: Parent has not been assigned a driver. System displays the message "Driver not assigned."
Business Rules	BR-1: Parents are only allowed to see the profiles of drivers assigned to their children. BR-2: Driver profile needs to be checked before it is displayed to Parents. BR-3: Mask contact information when necessary to privacy rules.
Assumptions	1. Parent is aware of which child's route to view.

	2. Parent is familiar with how to use the dashboard. 3. Device has the ability to display profile information. 4. Parent is able to read English.
--	---

Table 3.8: Track Child Location

Use Case ID	UC-8
Use Case Name	Track Child Location
Actors	Parent
Description	A Parent will be able to find out the real time location of their child at pickup/drop using the assigned driver's vehicle. This will keep the child safe and give an estimate of arrival time.
Trigger	When the Parent selects the 'Track Child Location' option from the dashboard or child's route details.
Pre-conditions	1. The Parent is signed in. 2. Child is registered and linked to the Parent's account. 3. Location of drivers' vehicle is tracked and working. 4. The system has GPS/location integration.
Post conditions	1. The Parent is able to see the child's current position displayed on a map. 2. Estimated pick up/drop off times are revised. 3. System logs the tracking activity for reference.
Normal Flow	1. The Parent clicks on 'Track Child Location' section. 2. The system fetches the real-time GPS data of the driver's vehicle. 3. As the child is displayed on the map, the system provides details of the route. 4. The Parent will be able to see the estimated time to be at the pick up/drop site. 5. On completion of the trip, the system will continuously update the location.
Alternative Flows	AF-1: Parent receives notifications when the child reaches pickup/drop points. AF-2: Parent views the last known location if real-time tracking is temporarily unavailable.
Exceptions	1.0. E1: GPS/location data unavailable. System shows "Location not available." 1.0. E2: Loss of device / internet connection. Asks the user to reconnect.

	1.0. E3: Child/Driver not assigned. The system shows that “Tracking not available”.
Business Rules	BR-1: Parents are only able to follow their own child. BR-2: The location data of the driver needs to be secured and encrypted. BR-3: Location updates should be as close to real time as possible to be accurate.
Assumptions	1. Parent gets hold of a device that has Internet and map display. 2. Driver’s vehicle has GPS tracking. 3. Parent is able to read English. 4. Users know how to use the tracking feature.

Table 3.9: Make Payment

Use Case ID	UC-9
Use Case Name	Make Payment
Actors	Parent
Description	A Parent can make payments for their child’s pick-up/drop services through the system. You can pay by a credit card, bank transfer or digital wallets. The payment is logged into the system and the child's service status is updated.
Trigger	When the Parent selects the ‘Make Payment’ option in the dashboard or billing section.
Preconditions	1. Parent is logged in. 2. Child registered and connected with Parent's account. 3. There is an integrated and working payment system. 4. Parent has adequate financial means or an acceptable form of payment.
Post conditions	1. Your payment was processed successfully. 2. Record of payment is kept in the system. 3. Parent/ Admin Driver are notified of payment. 4. The child's status is changed to "Active" for the paid time.
Normal Flow	1. The Parent enters ‘Make Payment’ section. 2. The system presents the list of invoices or payment methods that are pending. 3. Payments are made by the Parent for the child.

	4. The system checks the payment information. 5. The Parent acknowledges the payment. 6. The system handles the payment and records are updated. 7. A system is shown displaying a payment confirmation message and receipt.
Alternative Flows	AF-1: Parent schedules recurring payments. AF-2: Parent applies discounts, vouchers, or promo codes before payment. AF-3: Payment via external gateway (PayPal, Stripe, etc.).
Exceptions	1.0. E1: The payment has failed because of low funds. Displays the message, "Please try again." 1.0. E2: Payment information is not valid. System requests correction. 1.0. E3: Network or server error. Payment is not possible. 1.0. E4: Parent refuses to pay. No record of the transaction is kept.
Business Rules	BR-1: Payment must be related to right child and Parent account. BR-2: System should provide a receipt for every valid operation. BR-3: All payment information should be encrypted and secured. BR-4: Payments should be used to change the status of child services as soon as possible.
Assumptions	1. Parent has a valid payment method. 2. Parent is able to read English. 3. Parent is familiar with the process of online payments. 4. System's payment gateway is operational.

Table 3.10: Submit Feedback

Use Case ID	UC-10
Use Case Name	Submit Feedback
Actors	Parent
Description	A Parent may comment on the service or drivers or overall system experience. Feedback may be in the form of ratings, comments and suggestions that can help enhance the quality of service to the system.
Trigger	When the Parent selects the 'Submit Feedback' option in the dashboard or after a completed service.

Pre-conditions	1. The Parent is logged in. 2. The child has received pickup/drop services. 3. Feedback module is available and functional.
Post conditions	1. Feedback is recorded in the system. 2. Feedback is associated with the Parent, child and driver. 3. Admin and Drivers can see feedback to improve the service.
Normal Flow	1. The Parent clicks on 'Submit Feedback' section. 2. The system shows a form for rating (1 to 5 stars) and comments. 3. The Parent completes feedback information. 4. Parent gives the feedback. 5. The system stores the feedback in the database and acknowledges receipt of the feedback. 6. Feedback is available to the Admin/Driver.
Alternative Flows	AF-1: Parent submits only a rating without comments. AF-2: Parent edits feedback before final submission.
Exceptions	1.0. E1: Fields are empty. System prompts to fill them. 1.0. E2: Database error. Feedback will not be saved. 1.0. E3 Internet connection not available at submission time. System prompts retry.
Business Rules	BR-1: Every feedback should be attached to a particular Parent, child and service instance. BR-2: The stored feedback should be safe and cannot be modified by any unauthorized users. BR-3: Ratings should be within a valid range (eg 1-5).
Assumptions	1. Parent wants to give honest feedback. 2. Parent has the ability to read English. 3. Parent is aware of the rating/commenting process. 4. System is linked and working with database.

Table 3.11: Submit Verification Document

Use Case ID	UC-11
Use Case Name	Submit Verification Document
Actors	Driver
Description	A Driver may upload necessary documents required to verify (such as drivers license, ID, vehicle registration). This guarantees the Driver is

	authorized and verified prior to access to routes and child pick up/drop services.
Trigger	When the Driver selects the ‘Submit Verification Document’ option in their profile or dashboard.
Preconditions	1. The Driver is logged in. 2. The system supports document uploads. 3. Driver has scanned copies or photos of required documents.
Postconditions	1. Documents that are to be verified are uploaded and stored securely. 2. Admin receives notification – to verify documents. 3. Once the driver's account is verified, the status is updated.
Normal Flow	1. The Driver clicks on the ‘Submit Verification Document’ section. 2. Required document types and upload options are displayed. 3. Driver chooses which files to upload. 4. Formats and file sizes are checked. 5. The Driver hands over the documents. 6. System stores documents and notifies Admin for verification. 7. Driver confirms that submission has been received.
Alternative Flows	AF-1: Driver uploads documents one by one instead of all at once. AF-2: Driver updates previously submitted documents with new versions.
Exceptions	1.0. E1: Bad file format: Prompts for correct format to be uploaded on system. 1.0. E2: File size is too big. Please note that there are some system prompts to minimize file size. 1.0. E3: The network/server error prevents uploads. System prompts retry. 1.0. E4: Driver cancels upload. No documents are submitted.
Business Rules	BR-1: All submitted documents must be verified by Admin before Driver can access routes. BR-2: Documents must be stored securely and confidentially. BR-3: Drivers cannot perform pickup/drop until verification is approved.
Assumptions	1. Digital copies of documents are available for Driver to use. 2. Driver is familiar with the procedure for uploading files. 3. Device and internet connection support uploads. 4. Driver is able to read English.

Table 3.12: Mark Pick/Drop

Use Case ID	UC-12
Use Case Name	Mark Pick/Drop
Actors	Driver
Description	A Driver can check the status of each child when they are on their route (picked up or dropped off). This will make sure the Parents are updated in real time and the system is accurate.
Trigger	When the Driver reaches a pickup or drop-off location and selects the 'Mark Pick/Drop' option.
Preconditions	1. Driver is logged in. 2. Routes and children are assigned to the Driver. 3. Device/Application supports marking functionality. 4. Network or Internet connection is on.
Postconditions	1. Child's pickup/drop status is updated in the system. 2. Parents receive real-time notifications of status. 3. Route completion logs are updated for Admin.
Normal Flow	1. The Driver moves to the current section of the route. 2. Allows user to select a child from the list of children at the current stop. 3. The Driver chooses each child and makes a note of 'Picked Up' or 'Dropped Off.' 4. The system keeps the child status up-to-date. 5. Notifications are sent to Parents. 6. System records the action for Admin review.
Alternative Flows	AF-1: Driver marks more than one child at the same time. AF-2: Driver makes notes for each stop (e.g., Child absent/delay).
Exceptions	1.0. E1: Child Absent. System prompts to mark as absent. 1.0. E2: Network error. Retries on system prompts for status update. 1.0. E3: Driver does not make a marking. Status remains unchanged.
Business Rules	BR-1: Only assigned Drivers can assign children to their route. BR-2: Parents shall be informed immediately of status changes. BR-3: Status logs need to be securely managed for auditing purposes.
Assumptions	1. Driver understands which child corresponds to each pickup/drop. 2. Device supports marking functionality. 3. Internet connectivity is available. 4. Driver can read English.

Table 3.13: View Feedback and Ratings

Use Case ID	UC-13
Use Case Name	View Feedback and Ratings
Actors	Driver / Admin
Description	Parents can provide feedback and ratings on services, drivers and the system to be seen by Drivers and Admins. This enables to assess performance and enhance service quality.
Trigger	When the Driver or Admin selects the 'View Feedback and Ratings' option in the dashboard.
Preconditions	1. The user (Driver/Admin) is logged in. 2. Feedback has been submitted by Parents. 3. System stores feedback and rating data.
Postconditions	1. Any feedback and ratings are shown to the user. 2. System logs access for accountability. 3. Drivers/Admin can analyze trends or take actions based on feedback.
Normal Flow	1. User clicks on the 'View Feedback and Ratings' section. 2. The system fetches feedback and rating information that is pertinent to user. 3. Feed is presented in a "list" or "dashboard" format that includes detail information (e.g., Parent name, child, rating, comments, date). 4. User can sort, filter or search feedback by date, rating or child. 5. User reviews, and makes necessary actions.
Alternative Flows	AF-1: User generates reports or charts summarizing feedback. AF-2: User filters feedback for specific drivers (Admin only).
Exceptions	1.0. E1: No feedback to give. If the system says "No feedback found", it means that it was not able to identify any feedback. 1.0. E2: Database error. A message that displays in the system indicating that there's an error. 1.0. E3: User provides invalid filter. System prompts to correct.
Business Rules	BR-1: Drivers are only able to see feedback that is specifically related to their driving. BR-2: Admin can see feedback of all drivers/routes. BR-3: Feedback should not be changed to maintain integrity.

Assumptions	1. Users can read and interpret feedback. 2. Device supports feedback display. 3. System accurately links feedback to users. 4. Users can read English.
--------------------	--

Table 3.14: Manage Routes

Field	Description
Use Case ID	UC-14
Use Case Name	Manage Routes
Actors	Admin
Description	All routes are created, controlled and managed by an Admin. This encompasses driver allocation, stop planning, timing and reporting route status to ensure the safe, efficient transportation of students.
Trigger	When the Admin selects the 'Manage Routes' option from the Admin dashboard.
Preconditions	1. The Admin is logged in. 2. Drivers and vehicles exist in the system. 3. The system is connected to route and driver data.
Postconditions	1. The data in the routes is updated by the Admin. 2. Drivers and Parents can see updated route information. 3. All changes to the routes are recorded in the system.
Normal Flow	1. The Admin then clicks on the 'Manage Routes' section. 2. When system is displayed, all existing routes are shown. 3. Admin chooses an existing route or adds a new route. 4. Configure the Admin updates stops, timings, driver, or vehicle. 5. Admin saves changes. 6. System reflects changes to Drivers and Parents. 7. System acknowledges route management actions.
Alternative Flows	AF-1: Admin filters routes by date, driver, status, or vehicle. AF-2: Admin changes the driver of a route. AF-3: Admin installs temporary notices/instructions. AF-4: Admin copies an existing route to make a new one.
Exceptions	E1: Missing required fields. System validation error.

	E2: Database/server error. System reports an error message. E3: Admin cancels. Updates not saved. E4: Driver/vehicle scheduling conflict. Warning of conflict on system.
Business Rules	BR-1: Admin is able to add or edit any route. BR-2: Translations will need to be dynamic and update for Drivers and Parents. BR-3: No two drivers should be assigned the same routes. BR-4: Time changes on routes should be automatically notified to users.
Assumptions	1. Admin has an understanding of the transportation infrastructure. 2. Driver/vehicle data available and continually updated. 3. Admin device has route management interface. 4. Planning is possible with map/GPS data.

Table 3.15: Verify Driver

Use Case ID	UC-15
Use Case Name	Verify Driver
Actors	Admin
Description	Admin can confirm a Driver's account by checking on verification documents submitted. The Driver is then given a valid access to routes and pickup/drop services. This will keep them safe and adherent to standards.
Trigger	When the Admin selects the 'Verify Driver' option in the dashboard.
Preconditions	1. Admin is logged in. 2. Driver has submitted verification documents. 3. System stores submitted documents securely.
Postconditions	1. Driver account status is updated to 'Verified.' 2. Driver can access route assignments and pickup/drop features. 3. System logs the verification action.
Normal Flow	1. Admin navigates to the 'Verify Driver' section. 2. The system displays a list of Drivers pending verification. 3. Admin chooses a Driver to check a document. 4. Uploaded documents are reviewed by Admin. 5. Admin approves/denies verification. 6. Updates system and informs Driver.

	7. Verified Driver is now able to access all assigned functionalities.
Alternative Flows	AF-1: Admin requests additional documents if submitted ones are incomplete. AF-2: Admin temporarily marks Driver as pending verification until documents are clarified.
Exceptions	1.0. E1: No documents or documents are incorrect. Prompts Admin to refuse or ask for resubmission. 1.0. E2: Database error. There is not enough information to complete verification. 1.0. E3: Admin stops verification process. Driver remains unverified.
Business Rules	BR-1: Admin is only permitted to verify Drivers. BR-2: Drivers must be verified before accessing pickup/drop functionalities. BR-3: Verification logs must be securely stored for compliance purposes.
Assumptions	1. Admin is familiar with verification requirements. 2. Admin can read and evaluate documents. 3. The verification status is correctly connected to the Driver account. 4. User has the ability to read English.

Table 3.16: Manage Users

Use Case ID	UC-16
Use Case Name	Manage Users
Actors	Admin
Description	Admin can control all users of the system including Parents, Drivers, and other Admins. This includes adding, updating, deactivating, or deleting user accounts to ensure proper access control and system management.
Trigger	When the Admin selects the 'Manage Users' option in the dashboard.
Preconditions	1. Admin is logged in. 2. System stores user accounts and roles. 3. Admin has proper permissions to manage users.
Postconditions	1. Users accounts are added, updated or deleted based on Admin actions. 2. System keeps record of all user management events.

	3. The system for Access Control is updated with the updated information.
Normal Flow	1. Admin clicks on the ‘Manage Users’ section. 2. Users can see the list of all users including information (Name, Role, Status). 3. Admin chooses to Add, Update, Deactivate or Delete a user. 4. For Add: Admin fills in user details and submits. 5. For Update: Admin changes user data and saves changes. 6. For Deactivate/Delete: Admin confirms the action. 7. System makes changes, updates user's status and records that activity.
Alternative Flows	AF-1: Admin filters users by role (Parent/Driver/Admin). AF-2: admin can search for a user by name or ID. AF-3: When a user is disabled, it can be reactivated.
Exceptions	1.0. E1: Repetitive user ID/email. System prompts to correct. 1.0. E2: Database error. No changes can be made. 1.0. E3: Admin cancels action. There are no modifications made.
Business Rules	BR-1: Users can only be managed by Admin. BR-2: Access is limited by user roles. BR-3: All user operations for management of the system should be recorded for accountability. BR-4: If a user is deactivated, he/she will be unable to log in until reactivation.
Assumptions	1. Admin has an understanding of user roles and permissions. 2. System interface makes it easy to navigate. 3. Admin can read English. 4. Personal information of users is kept safe.

Table 3.17: View Parent and Driver Payments

Use Case ID	UC-17
Use Case Name	View Parent and Driver Payments
Actors	Admin
Description	Admin will be able to see the payment details from Parents and payments made to Drivers. This makes money management and

	maintaining precise transactions easier and can help resolve any payment issues.
Trigger	When the Admin selects the ‘View Payments’ option in the dashboard.
Preconditions	1. Admin is logged in. 2. Payment records are stored in the system. 3. System is connected to the financial module.
Postconditions	1. Payment details are displayed to Admin. 2. System logs access for auditing. 3. Admin can verify transactions, generate reports, or identify discrepancies.
Normal Flow	1. Admin navigates to the ‘View Payments’ section. 2. The system retrieves payment records for Parents and Drivers. 3. Admin can see payment information like amount, date, service, payee/payer. 4. Admin can run a filter or a sort based on date, child, driver or service. 5. Admin can create reports for financial tracking or audit.
Alternative Flows	AF-1: Admin exports payment data to CSV, Excel, or PDF. AF-2: Admin filters payments for a specific period or user.
Exceptions	1.0. E1: No payment records found. System in the status shows no payments. 1.0. E2: Database error. Payment information is not recoverable. 1.0. E3: Admin abandons report generation. No data exported.
Business Rules	BR-1: All payment records will only be available to view by Admin. BR-2: Payment data must be accurate and correspond to recorded services. BR-3: All financial data should be secured and encrypted. BR-4: System must track all access and report generation for audit purposes.
Assumptions	1. Admin has an understanding of financial documents. 2. Device can display and export payment information. 3. System properly integrates payments with users. 4. Admin can read English.

Table 3.18: Manage Admins

Use Case ID	UC-18
--------------------	--------------

Use Case Name	Manage Admins
Actors	Super Admin
Description	Add, update, deactivate and delete Admin users - Super Admin can manage Admin accounts and add, update, deactivate or delete Admin users. This provides correct management and administration of a system.
Trigger	When the Super Admin selects the 'Manage Admins' option in the dashboard.
Preconditions	1. Super Admin is logged in. 2. System stores Admin accounts and roles. 3. Super Admin has permissions to manage Admin users.
Postconditions	1. Admin accounts are automatically generated, updated or deleted based on super admin actions. 2. System records all actions by Administrator in the Management. 3. The system has been updated with new Admin access.
Normal Flow	1. Super Admin navigates to the 'Manage Admins' section. 2. The system displays a list of Admin accounts with details (name, role, status). 3. Super Admin chooses Add, Update, Deactivate, Delete Admin. 4. For Add: Super Admin completes information and submits. 5. For Update: Super Admin modifies Admin information and saves changes. 6. For Deactivate/Delete: When performing the action Deactivate/Delete, the Super Admin confirms the action. 7. System applies changes, updates Admin status, logs the activity.
Alternative Flows	AF-1: Super Admin filters Admins by status or role. AF-2: Super Admin searches for an particular Admin by his/her name or ID. AF-3: Super admin re-enables a previously disabled Admin.
Exceptions	1.0. E1: Duplicate Admin ID/Email. System prompts to correct. 1.0. E2: Database error. Changes will not be applied. 1.0. E3: Super Admin cancels action. No changes occur.
Business Rules	BR-1: Only Super Admin can deal with Admin accounts. BR-2: Access levels are determined by Admin roles.

	BR-3: System must log all Admin management actions for accountability. BR-4: Admins who have been de-activated will not be able to sign in until they are reactivated.
Assumptions	1. Super Admin is familiar with Admin roles and permissions. 2. Device has a user friendly navigation and management interface. 3. Super Admin can read English. 4. There is security over admin data.

Table 3.19: Manage Payments

Use Case ID	UC-19
Use Case Name	Manage Payments
Actors	Super Admin
Description	Super Admin handles all financial activities such as receiving payments from Parents, and payments to Drivers and Admins. This will guarantee that all transactions are accurately recorded, tracked, and distributed within the system.
Trigger	When the Super Admin selects the 'Manage Payments' option in the dashboard or financial module.
Preconditions	1. Super Admin is logged in. 2. Payment module is functional. 3. Parents have initiated payments. 4. Earnings for Drivers/Admins are calculated.
Postconditions	1. Payments are recorded and updated in the system. 2. Payments are distributed to Drivers/Admins. 3. Financial records are updated and stored securely. 4. Notifications are sent to all relevant users.
Normal Flow	1. Super Admin goes to the 'Manage Payments' section. 2. System shows payments received from Parents. 3. Super Admin confirms and verifies the receipts of payments. 4. Updates payment records system. 5. Pending payouts for Drivers/Admins is displayed in the system. 6. Super Admin checks payouts. 7. Super Admin has confirmed payment distribution. 8. Updates the records and notifies.
Alternative Flows	AF-1: Super Admin processes payments in batches. AF-2: Super Admin schedules future distributions. AF-3: Super Admin adjusts payments manually (e.g., bonuses or corrections).
Exceptions	E1: Payment failed because of a network or system error → retry option.

	E2: Payment amount mismatch – flagged for review. E3: Super Admin cancels transaction → pending.
Business Rules	BR-1: Only Super Admin can manage payments. BR-2: All payments must be accurately recorded and linked to users. BR-3: Ensure financial records are kept securely. BR-4: Notifications should be sent post transactions.
Assumptions	1. Super Admin has financial knowledge. 2. System can accept payments. 3. Device has a payment interface. 4. The Super Admin is able to read English.

Table 3.20: Logout

Use Case ID	UC-20
Use Case Name	Logout
Actors	User (Parent / Driver / Admin)
Description	Any logged-in user can securely log out of the system to end their session, protecting personal data and preventing unauthorized access.
Trigger	When the user selects the ‘Logout’ option in the application or website.
Preconditions	1. User is logged in. 2. System session is active. 3. Internet connection is available.
Postconditions	1. User session is terminated. 2. User is redirected to the login or home page. 3. System logs the logout event for security purposes.
Normal Flow	1. User clicks on the ‘Logout’ button. 2. The system verifies the action (optional: "Are you sure?" dialog box). 3. Session is ended and Session Data is cleared. 4. User is taken to the login/home page. 5. Logout event is recorded in system logs.
Alternative Flows	AF-1: User cancels the logout confirmation; session remains active. AF-2: System automatically logs out user after inactivity timeout.
Exceptions	1.0. E1: Network or server error prevents logout. System prompts retry. 1.0. E2: Session already expired. System redirects to login page.
Business Rules	BR-1: Securely log off users. BR-2: Security and audit logs of all logout events. BR-3: Session data shall be wiped so that it cannot be accessed by others.

Assumptions	1. User understands the logout process. 2. Device supports session management. 3. Internet connection is available during logout. 4. User can read English.
--------------------	--

3.3 Functional Requirements

Table 3.21: Sign up FR-1

Identifier	FR-1
Title	Signup
Requirement	The user (Parent or Driver) shall be able to create an account to access the system and manage relevant data.
Source	System Analyst / Stakeholder
Rationale	Accounts allow users to securely save and retrieve personal data and access role-specific features.
Business Rule	Email must be unique for each account.
Dependencies	N/A
Priority	High

Table 3.22: Login FR-2

Identifier	FR-2
Title	Log In
Requirement	After registration, the user shall be able to log in using their credentials to access the system.
Source	System Analyst / Stakeholder
Rationale	Provides authenticated access to the system and protects user data.
Business Rule	N/A
Dependencies	FR-1
Priority	High

Table 3.23: Manage Profile FR-3

Identifier	FR-3
Title	Manage Profile
Requirement	The user (Parent or Driver) shall be able to update their personal profile.
Source	Stakeholder / User
Rationale	Keeps user information accurate and allows personalization of services.

Business Rule	N/A
Dependencies	FR-1, FR-2
Priority	High

Table 3.24: View Routes FR-4

Identifier	FR-4
Title	View Routes
Requirement	The user (Parent or Driver) shall be able to view assigned routes including pickup/drop locations, timings, and driver/child details.
Source	Stakeholder / User
Rationale	Ensures clarity on route assignments and planning for pickups and drops.
Business Rule	Only authorized users can access route details.
Dependencies	FR-1, FR-2
Priority	High

Table 3.25: Add Child FR-5

Identifier	FR-5
Title	Add Child
Requirement	The Parent shall have the capability to enter the name, date of birth, class, pick up and drop information of the child on the system for tracking and service management.
Source	Stakeholder / Parent
Rationale	Allows the system to manage multiple children per parent and assign routes correctly.
Business Rule	A child must be linked to only one Parent account.
Dependencies	FR-1, FR-2
Priority	High

Table 3.26: View Notifications FR-6

Identifier	FR-6
Title	View Notifications
Requirement	The Parent shall be able to view system notifications related to route changes and pickup/drop updates
Source	Stakeholder / Parent
Rationale	Keeps Parents informed about important updates and ensures timely actions.
Business Rule	Notifications must be displayed in chronological order with the latest first.
Dependencies	FR-1, FR-2
Priority	High

Table 3.27: View Driver Profile FR-7

Identifier	FR-7
Title	View Driver Profile
Requirement	The Parent shall be able to view the assigned Driver's profile including name, contact info, rating, feedback, and profile picture .
Source	Stakeholder / Parent
Rationale	Builds trust and transparency for Parents regarding their child's Driver.
Business Rule	Only authorized Parent can view their child's Driver profile.
Dependencies	FR-1, FR-2, FR-5
Priority	High

Table 3.28: Track Child Location FR-8

Identifier	FR-8
Title	Track Child Location
Requirement	The Parent shall be able to track the real-time location of their child during pickup/drop via the system.
Source	Stakeholder / Parent
Rationale	Ensures child safety and provides peace of mind to Parents.
Business Rule	Only authorized Parent can track their child.
Dependencies	FR-1, FR-2, FR-5

Priority	High
-----------------	------

Table 3.29: Make Payment FR-9

Identifier	FR-9
Title	Make Payment
Requirement	The Parent shall be able to make payments for pickup/drop services using integrated payment methods (card, bank, or digital wallet).
Source	Stakeholder / Parent
Rationale	Ensures timely and secure collection of fees.
Business Rule	Payment must be confirmed before service completion.
Dependencies	FR-1, FR-2, FR-5
Priority	High

Table 3.30: Submit Feedback FR-10

Identifier	FR-10
Title	Submit Feedback
Requirement	The Parent shall be able to submit feedback regarding the service, driver behavior, or system issues through the application.
Source	Stakeholder / Parent
Rationale	Collecting feedback helps improve service quality and driver performance.
Business Rule	Feedback must be linked to the specific driver or service instance.
Dependencies	FR-1, FR-2, FR-5
Priority	High

Table 3.31: Submit Verification Document FR-11

Identifier	FR-11
Title	Submit Verification Document
Requirement	The Driver shall be able to upload required verification documents (e.g., license, CNIC, vehicle papers) through the system for approval.
Source	Stakeholder / Driver

Rationale	Ensures that only verified Drivers provide pickup/drop services, maintaining safety and compliance.
Business Rule	Driver cannot access route or pick/drop features until verification is approved.
Dependencies	FR-1, FR-2
Priority	High

Table 3.32: Mark Pick/Drop FR-12

Identifier	FR-12
Title	Mark Pick/Drop
Requirement	The Driver shall have the ability to enter the time and status of each child picked up and/or dropped off, to be tracked.
Source	Stakeholder / Driver
Rationale	Ensures accurate tracking of child pickups and drops for safety and accountability.
Business Rule	Only assigned Drivers can mark children on their specific routes.
Dependencies	FR-1, FR-2, FR-12
Priority	High

Table 3.33: Update Availability FR-13

Identifier	FR-13
Title	Update Availability
Requirement	The Driver shall be able to update their availability status in the system (e.g., available, unavailable, on leave) for scheduling purposes.
Source	Stakeholder / Driver
Rationale	Helps the system manage route assignments and ensures that only available Drivers are scheduled for pickups and drops.
Business Rule	Availability changes must be reflected in real-time and visible to the system and Admins.
Dependencies	FR-1, FR-2
Priority	High

Table 3.34: View Feedback and Ratings FR-14

Identifier	FR-14
Title	View Feedback and Ratings
Requirement	The Driver or Admin shall be able to view feedback and ratings submitted by Parents regarding service quality, driver behavior, and overall satisfaction.
Source	Stakeholder / Driver / Admin
Rationale	Helps Drivers improve performance and allows Admins to monitor service quality.
Business Rule	Only authorized Drivers can view feedback related to their routes; Admins can view feedback for all Drivers.
Dependencies	FR-1, FR-2, FR-10
Priority	High

Table 3.35: View Earnings FR-15

Identifier	FR-15
Title	View Earnings
Requirement	The Driver or Admin shall be able to view their earnings, including payments received, pending payments, and payment history.
Source	Stakeholder / Driver / Admin
Rationale	Provides transparency and allows Drivers/Admins to track financial records and payments.
Business Rule	Earnings must be calculated accurately based on completed pickups/drops and confirmed payments.
Dependencies	FR-1, FR-2, FR-9, FR-24
Priority	High

Table 3.36: Manage Routes FR-16

Identifier	FR-16
Title	Manage Routes
Requirement	The Admin must be able to edit, create, modify and delete pickup and drop routes specific to the system, assign drivers, edit stops, edit timing and manage child list for each route.
Source	Stakeholder / Admin

Rationale	Ensures centralized control and proper management of all transportation routes across the school/college system. Helps maintain accuracy, safety, and efficient transportation scheduling.
Business Rule	Only authorized Admins can modify or delete any route. Drivers can only edit their assigned routes.
Dependencies	FR-1, FR-2, FR-1
Priority	High

Table 3.37: Verify Driver FR-17

Identifier	FR-17
Title	Verify Driver
Requirement	The Admin shall be able to verify driver-submitted documents (license, CNIC, vehicle papers) and approve or reject the Driver's account for system access.
Source	Stakeholder / Admin
Rationale	Ensures that only authorized and verified Drivers provide pickup/drop services, maintaining safety and compliance.
Business Rule	A Driver cannot access route or pickup/drop features until verification is approved.
Dependencies	FR-1, FR-2, FR-11
Priority	High

Table 3.38: Manage Users FR-18

Identifier	FR-18
Title	Manage Users
Requirement	The Admin shall be able to add, update, deactivate, or delete Parent and Driver accounts through the system for proper user management.
Source	Stakeholder / Admin
Rationale	Ensures that the system has accurate user records and allows administrative control over user access.
Business Rule	Only Admins with proper authorization can manage user accounts; deactivated users cannot access the system.
Dependencies	FR-1, FR-2, FR-18
Priority	High

Table 3.39: Manage Payments FR-19

Identifier	FR-19
Title	Manage Payments
Requirement	Super Admin should be able to perform all activities related to payment, such as accepting and logging payments from parents and allocating payments to Drivers and Admins as per their earnings in the system (using integrated payment systems like card, bank transfer or digital wallet).
Source	Stakeholder / Super Admin
Rationale	Ensures secure, accurate, and efficient handling of all financial transactions, including collection and distribution of payments within the system.
Business Rule	All payments to be verified and recorded prior to distribution. Payment can only be managed and distributed by the Super Admin. All deals must be recorded for audit.
Dependencies	FR-1, FR-2, FR-9
Priority	High

Table 3.40: Logout FR-20

Identifier	FR-20
Title	Logout
Requirement	Any logged-in user (Parent, Driver, Admin, Super Admin) shall be able to securely log out of the system to terminate their session.
Source	Stakeholder / System Analyst
Rationale	Prevents unauthorized access and protects user data, especially on shared or public devices.
Business Rule	Session data must be cleared upon logout; logout events must be recorded in system logs.
Dependencies	FR-1, FR-2
Priority	High

3.4 Non-Functional Requirements

3.4.1 Usability

USE-1: The system shall provide a simple, clean, and easy-to-navigate interface for all user roles.

3.4.2 Performance

PER-1: Dashboards and route data shall load within 3 seconds on a standard 4G connection.

3.4.3 Security

SEC-1: Sensitive information shall be protected through role-based access and Firebase security rules.

3.4.4 Maintainability

MAI-1: The codebase shall follow a modular structure to allow adding or updating features without affecting existing modules.

4. Design and Architecture

The overall structure of School/College Pick And Drop Management System is described in this chapter. System is designed by the layered approach, which brings the following benefits: Scalability, Maintainability, and Efficient performance. Compromising each one of these components are separated layers, which are organized and linked together. This allows them to update independently without impacting the system. The design also allows for real-time capabilities, such as route tracking, communication and system operations.

4.1 Architectural design

It relies on a multi-layered service-oriented architecture that facilitates vital functions like driver management, route tracking, payment processing and communication. The user interface layer offers interaction via web and mobile app for Super Admin, Admin, Parent and Driver. All the operations are applied and the rules of the system are implemented in the business logic layer. It is made of modules such as route management, chat, payments and so on. Data access layer is used to control the operations of structured data and as a bridge between the database and logic. To ensure secure data handling, the data layer is implemented using Firebase services, namely authentication and database storage. There is integration for external services like Google Maps API for live tracking and navigation. Each layer communicates with the others in a well-defined manner, with user requests being efficiently processed, stored, and returned, ensuring smooth and reliable system performance. Figure 4.1 illustrates the system architecture of School/College Pick and Drop Management System.

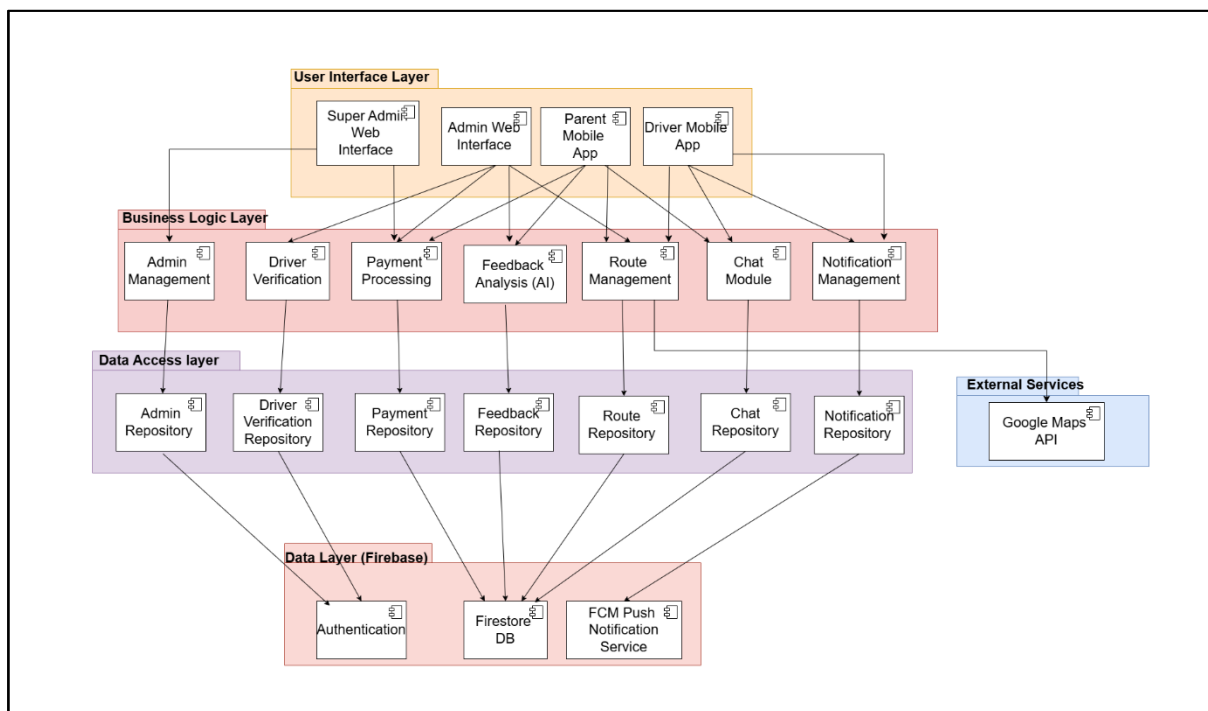


Figure 4.1 : Architecture Diagram

4.2 Process flow/Representation

The activity diagrams representing the main processes of School/College Pick and Drop Management System for the roles of the Super Admin, Admin, Parent and Driver are depicted in Figure 4.2, 4.3, 4.4 and 4.5. This is achieved by user authentication that grants access to the user to the role-based dashboards. Super Admin is responsible for the Admins and payment activities. At the same time, Admin processes the drivers, parents and feedback in several operations, which can occur simultaneously. Parents are able to view routes, send messages to drivers, and be alerted. Drivers can handle routes, update trip status and communicate with parents. The workflows demonstrate the system's ability to support real-time interaction, role-specific functionality, and seamless task completion, creating a cohesive and coordinated user experience.

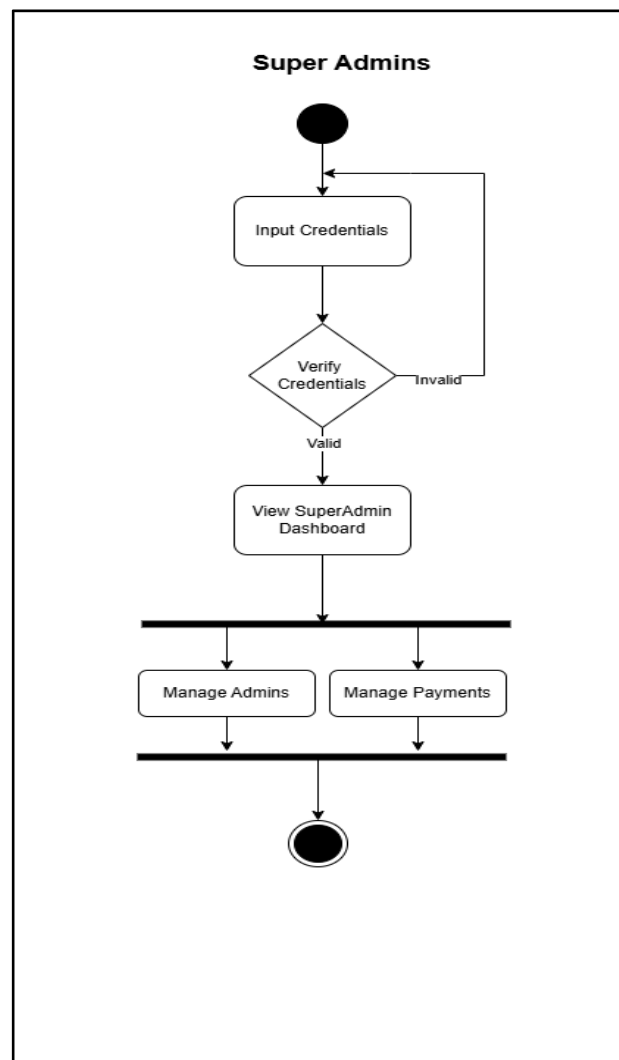


Figure 4.2 : Activity Diagram of Super Admin

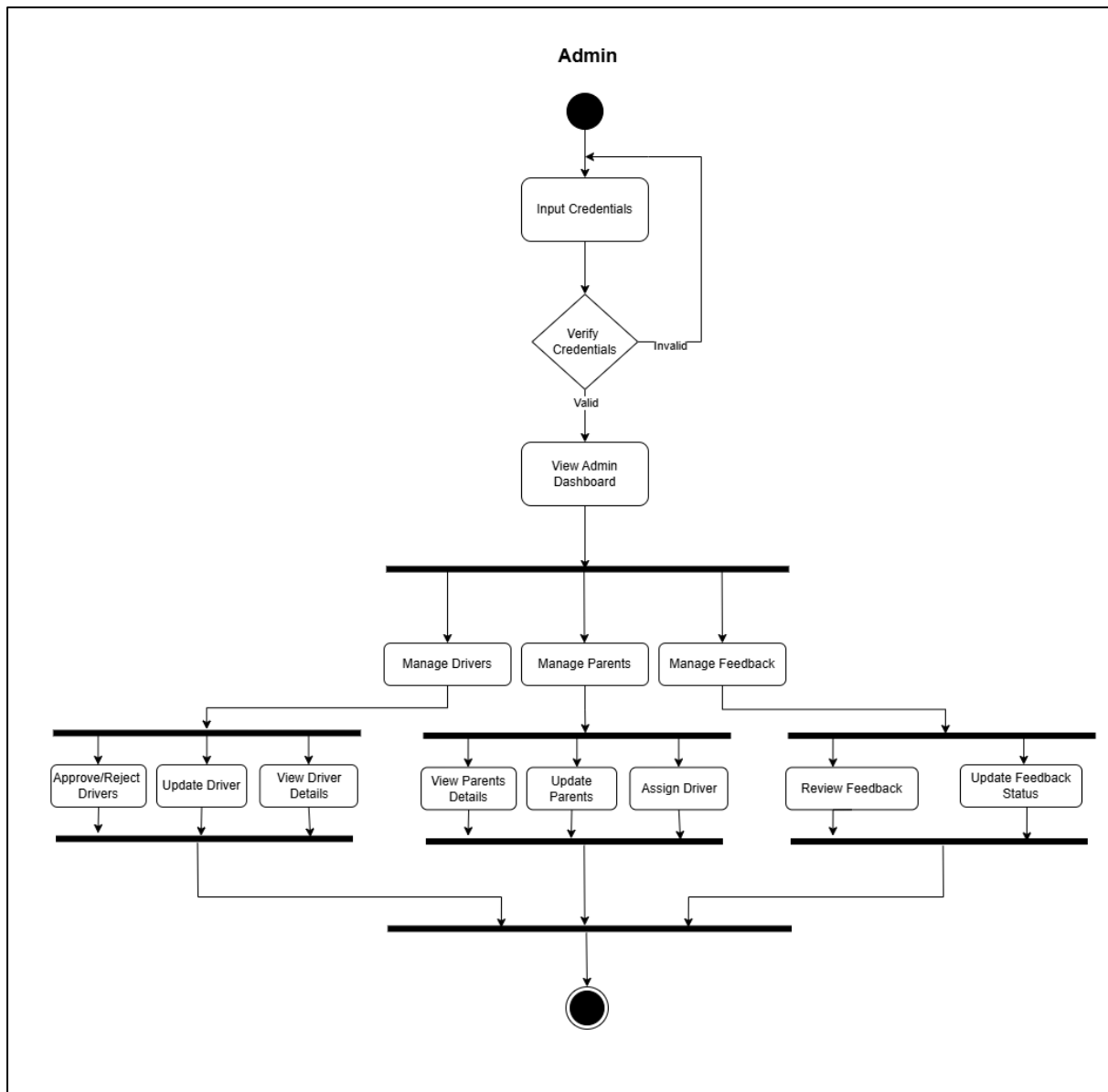


Figure 4.3 : Activity Diagram of Admin

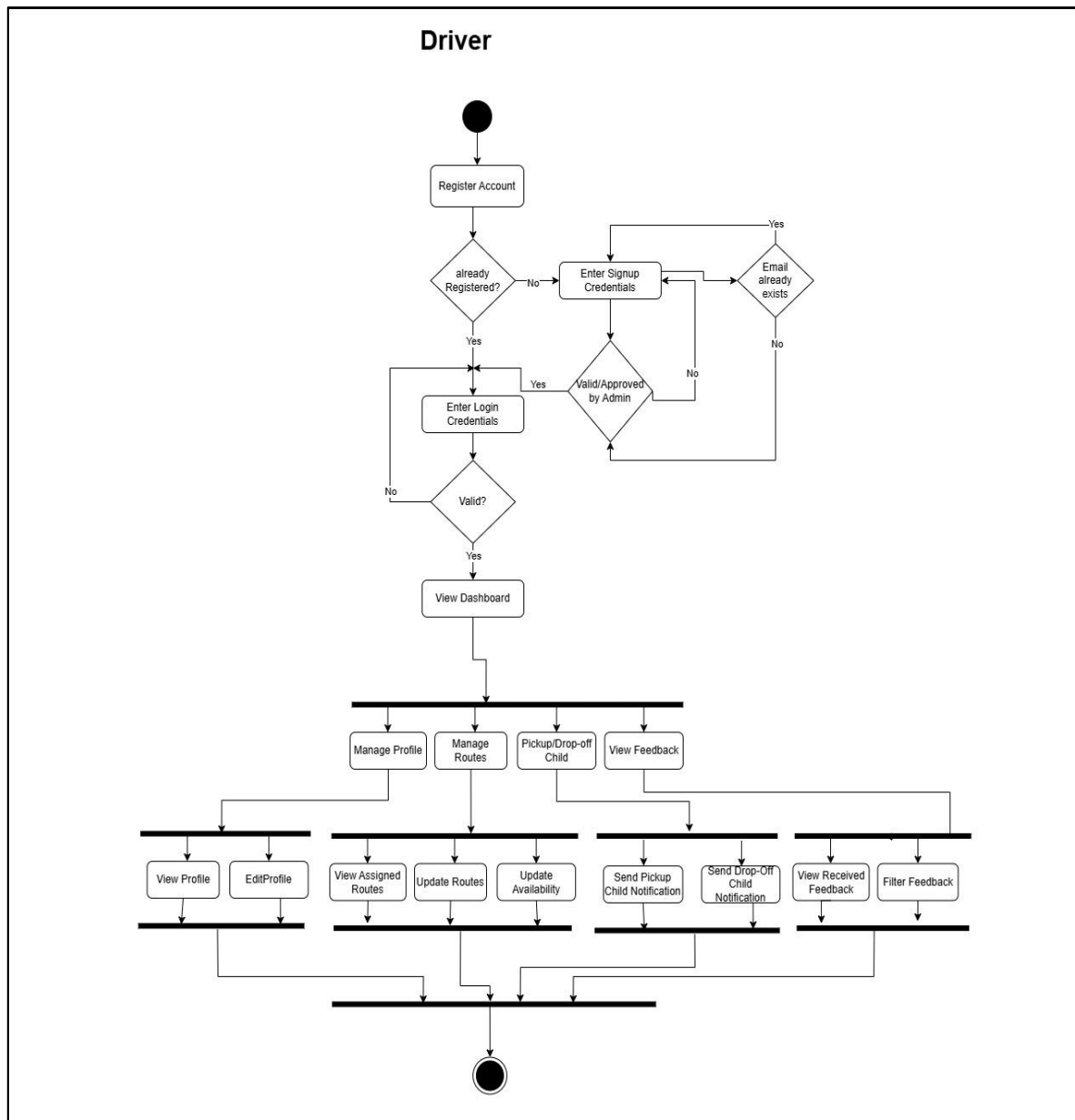


Figure 4.4 : Activity Diagram of Driver

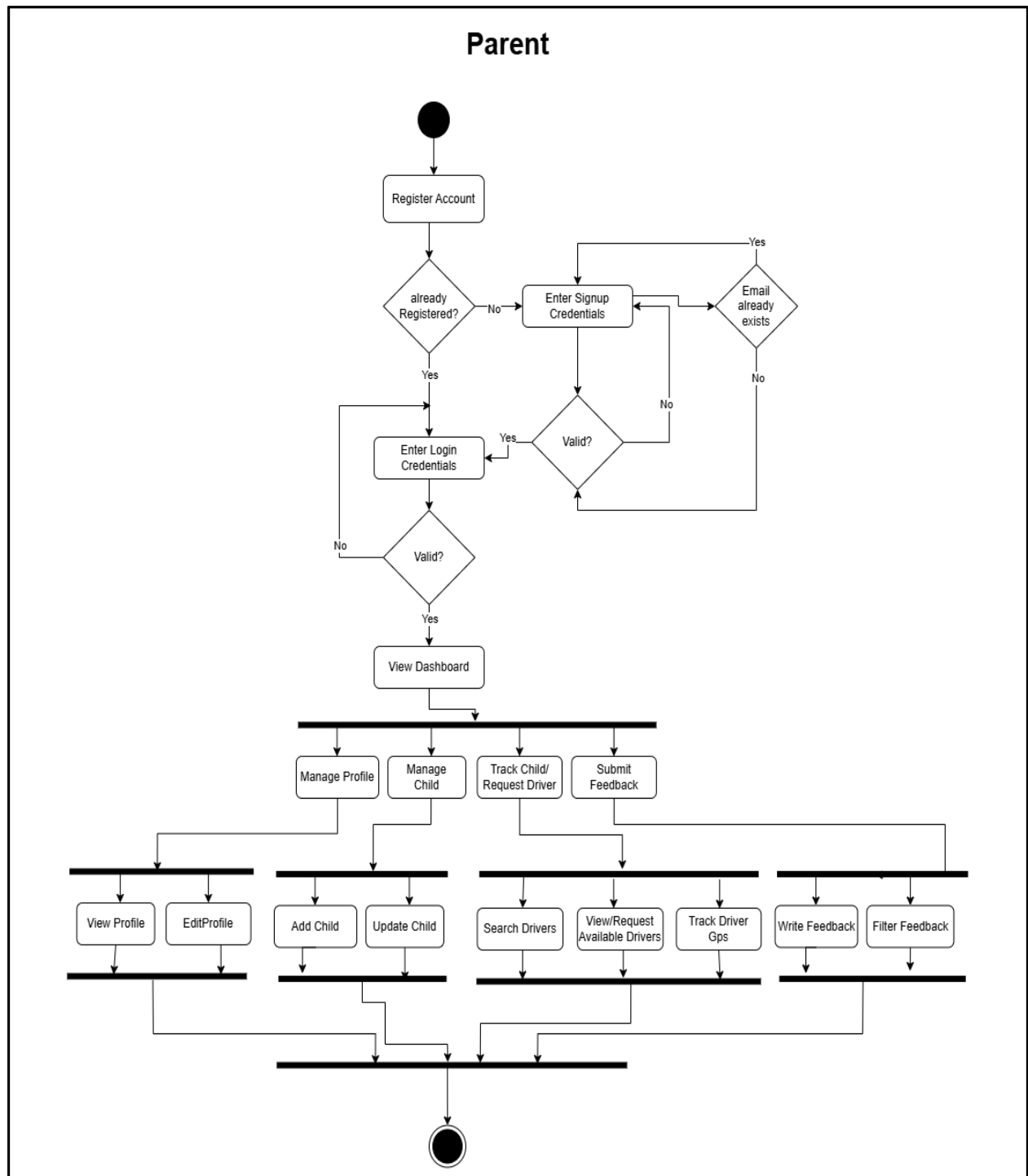


Figure 4.5: Activity Diagram of Parent

4.2.1 Json Tree

Figure 4.6 represents a hierarchical structure in JSON (JavaScript Object Notation) format of the Pick and Drop Management System in the School/College. The figure 4.6 is the hierarchical JSON (JavaScript Object Notation) tree structure of the Pick and Drop Management System in the School/College where the data is stored and retrieved efficiently. The structure is based on the main root node "PickAndDropSystem", which has main entities, such as Super Admin, Admin, Parent and Driver. The Super Admin node will be used to set the system level settings and admin control. The Admin node is where all the

operational elements such as driver management, parents and routes are managed. The Parent node stores parent and child, pickup request/notification information, and the Driver node stores driver, vehicle, assigned routes and trip record information. Other parts are pick-up requests, trip management, feedback and system operations and monitoring of real-time tracking.

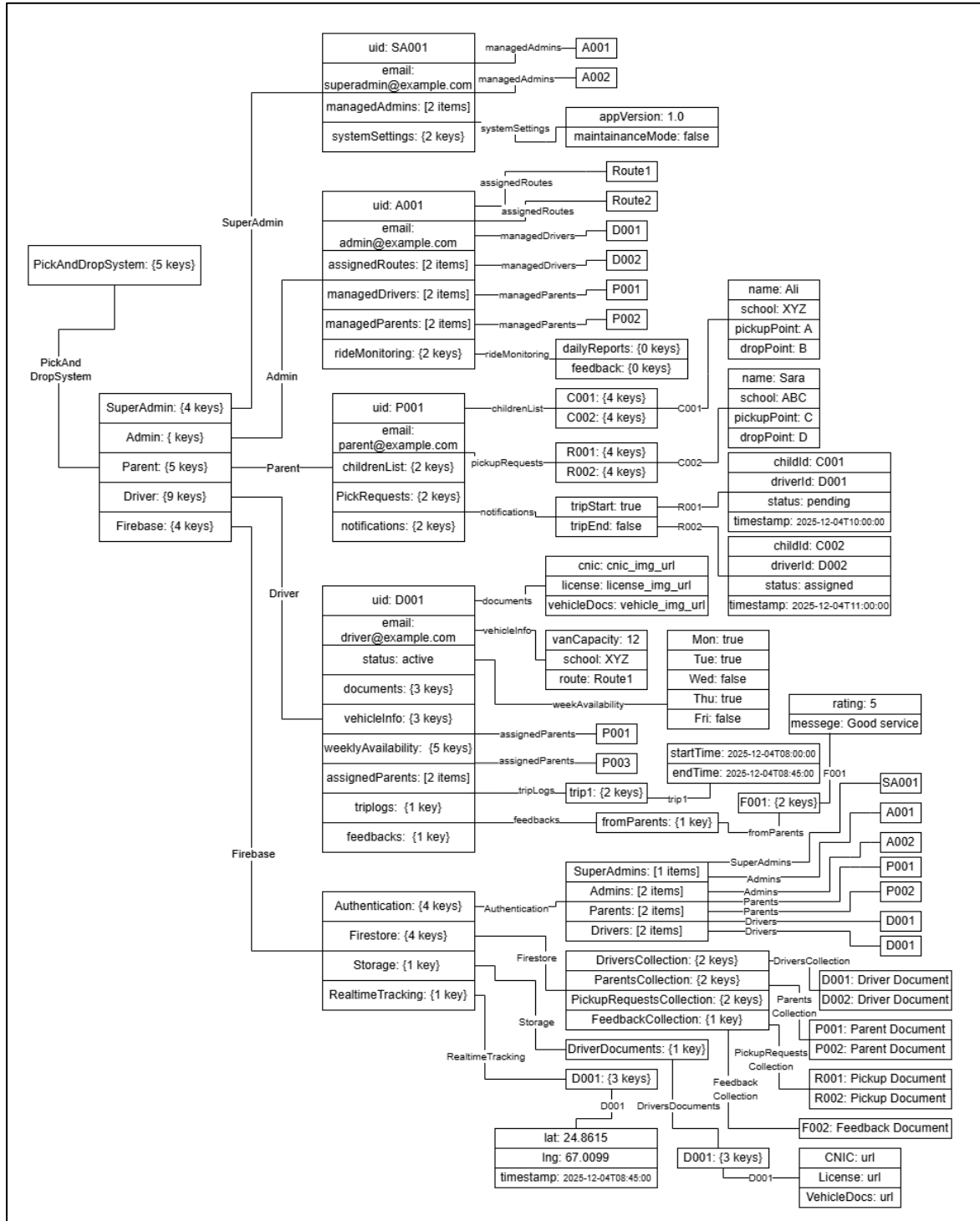


Figure 4.6 : Json Tree

4.3 Design Models

Design models: Organized structure and behaviour of the system prior to implementation. These are the models that convert functional requirements into technical representations that specify the components of the system, their function and interaction. Design models are used in this project to represent the major entities of the system, the flow of data in and out of these entities and the order in which operations will be carried out across various processes. The primary models are Class diagrams and the corresponding Sequence Diagrams which make it easy to see the architecture of the system and the communication patterns.

4.3.1 Class Diagram

Figure 4.7 depicts the class diagram of School/College Pick and Drop Management System which represents the static structure of the system. It is based on a main class User, having common attributes like userid, name, email, phone number and password, login and logout functions. It is extended by SuperAdmin class, Admin class, Parent class and Driver class where each class has its own responsibility to manage the system, routes and users, to accommodate the transportation of children and driving. Other significant classes are Child for details of the students, Route for the definition of the travel paths, Notification for system messages between users. TrackingService class is used to track real time tracking of child during transport. Overall the diagram illustrates the interactions and collaborations between the various classes, and how they relate to each other through inheritance and association to implement the complete pick and drop system.

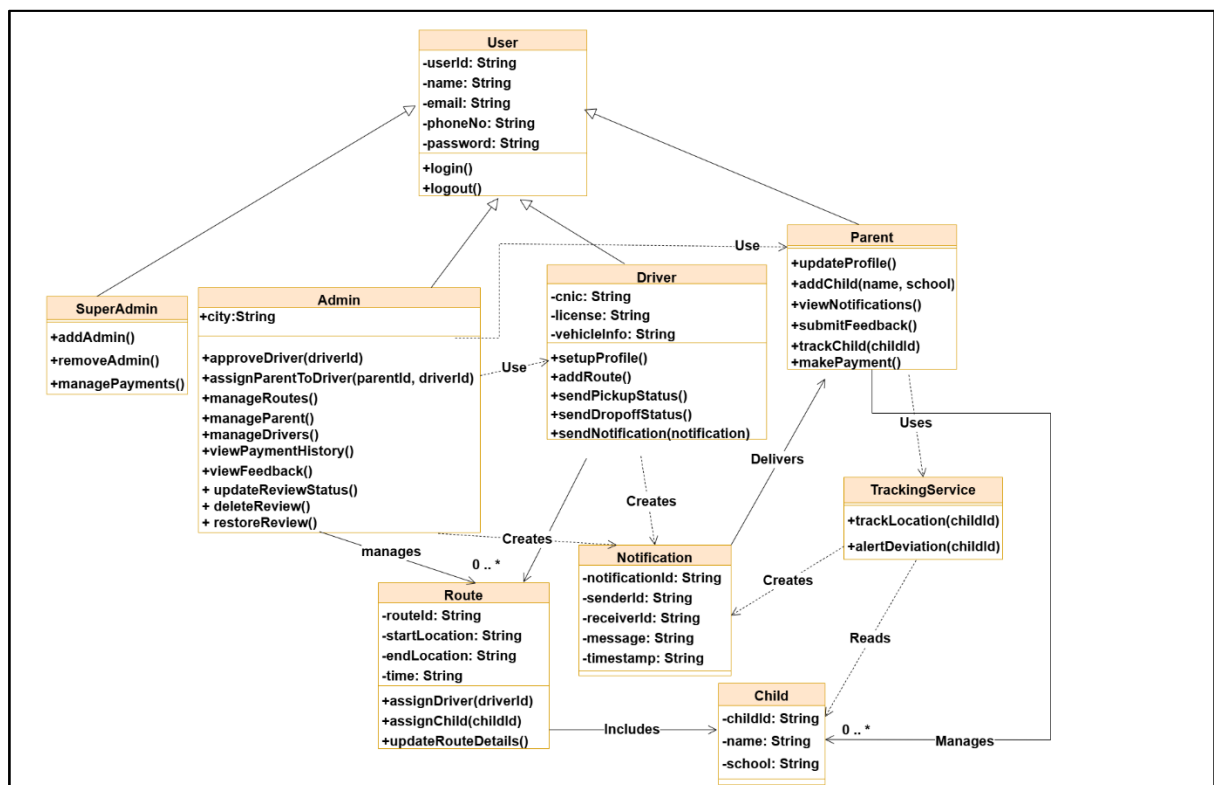


Figure 4.7: Class Diagram of Pick and Drop Management System

4.3.2 Sequence Diagram

The step-by-step interaction between the users and the system during various operations are presented in the sequence diagrams of the School/College Pick and Drop Management System. There are four main sequence diagrams; one for each module: Super Admin, Admin, Driver, and Parent, showing the flow of requests processed and the generation of responses from various services. Super Admin manages the admin accounts and keeps track of the payments. Admin will handle the primary functions such as driver approval, route management, assigning parents, feedback and processing payments. The driver sequence is around making profiles, managing routes, starting trips, verifying children and sending out pickup/ drop notifications – ensuring safe and smooth transportation. Parent sequence is the process of registering parents, monitoring the transportation of children, informing, payment and feedback. To summarize, these sequence diagrams offer a structured and clear view of the communication between different modules, and the behavior of the system.

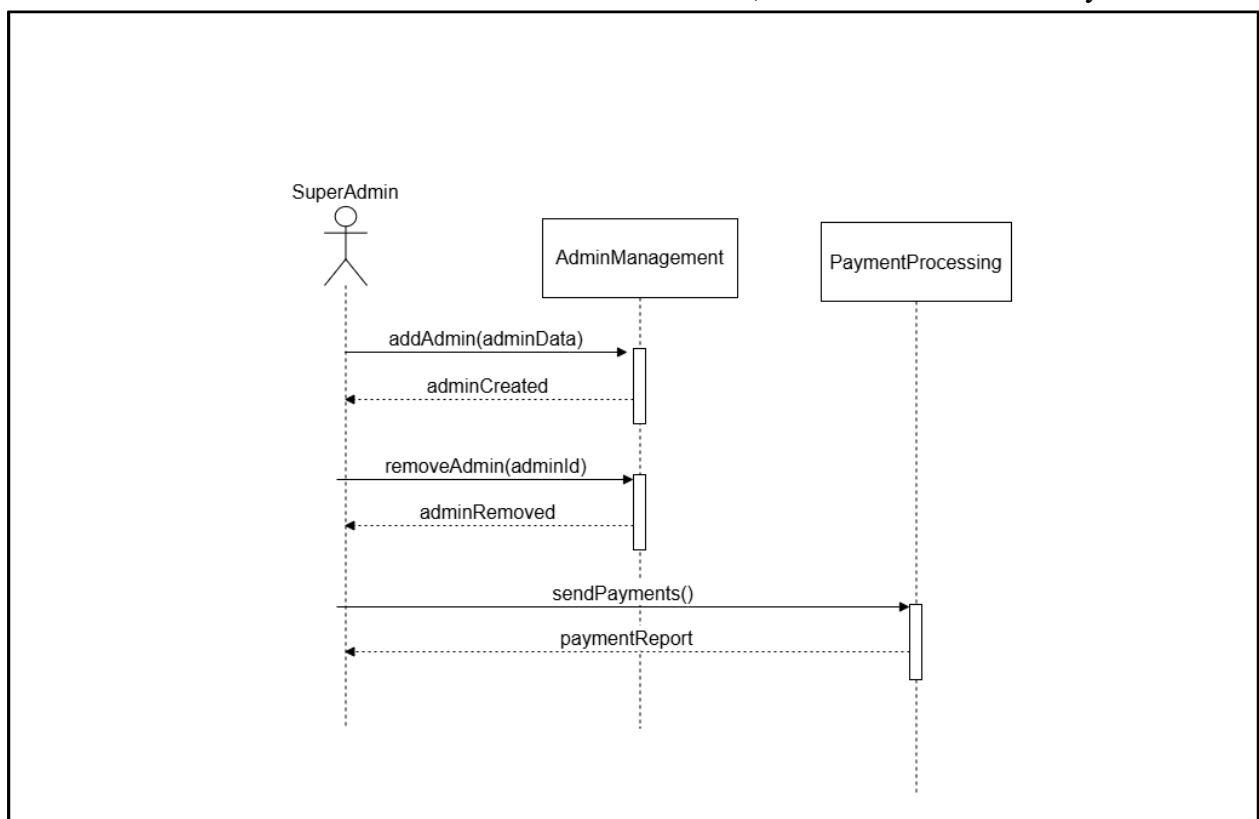


Figure 4.8: Sequence Diagram of Super Admin

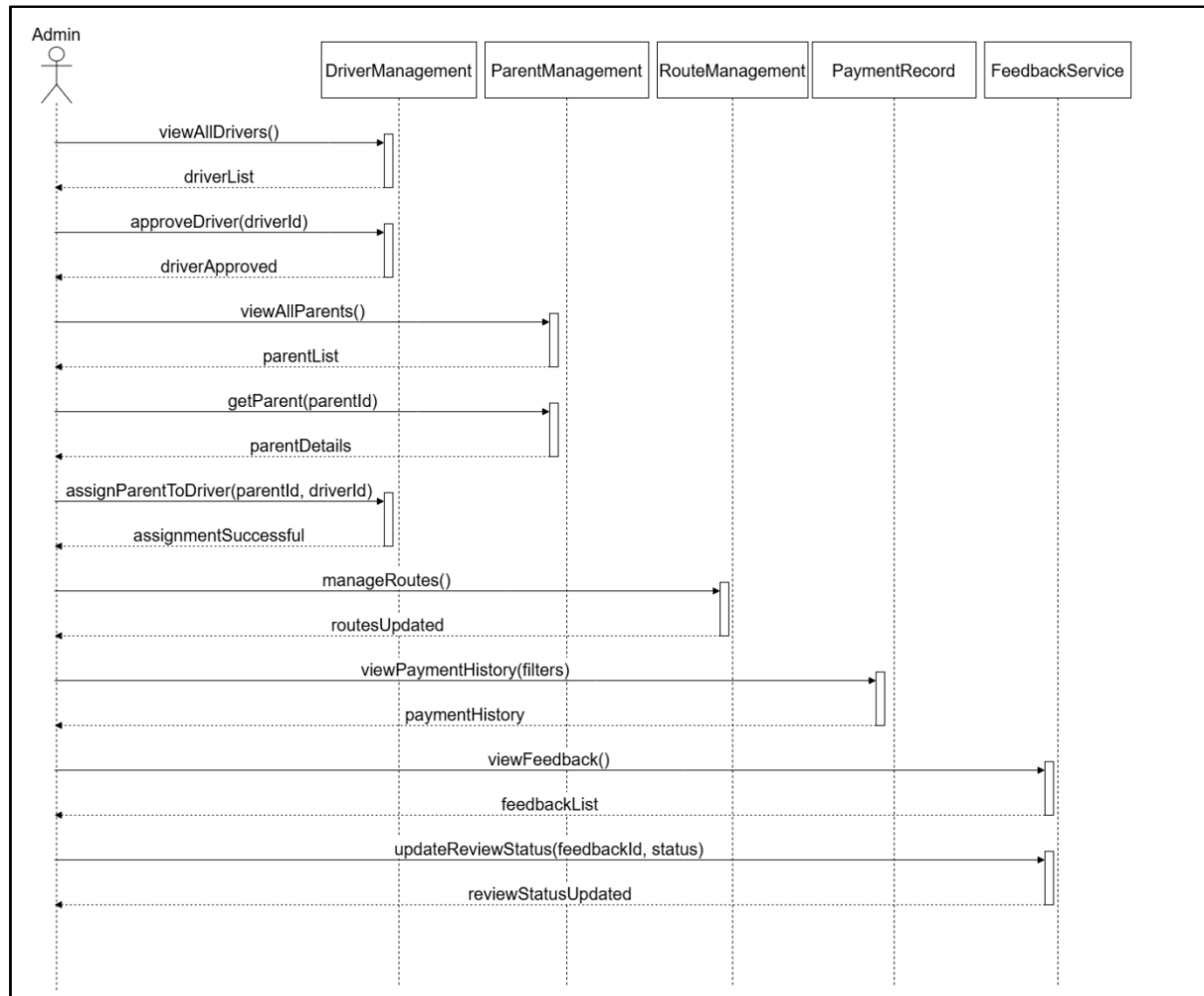


Figure 4.9: Sequence Diagram of Admin

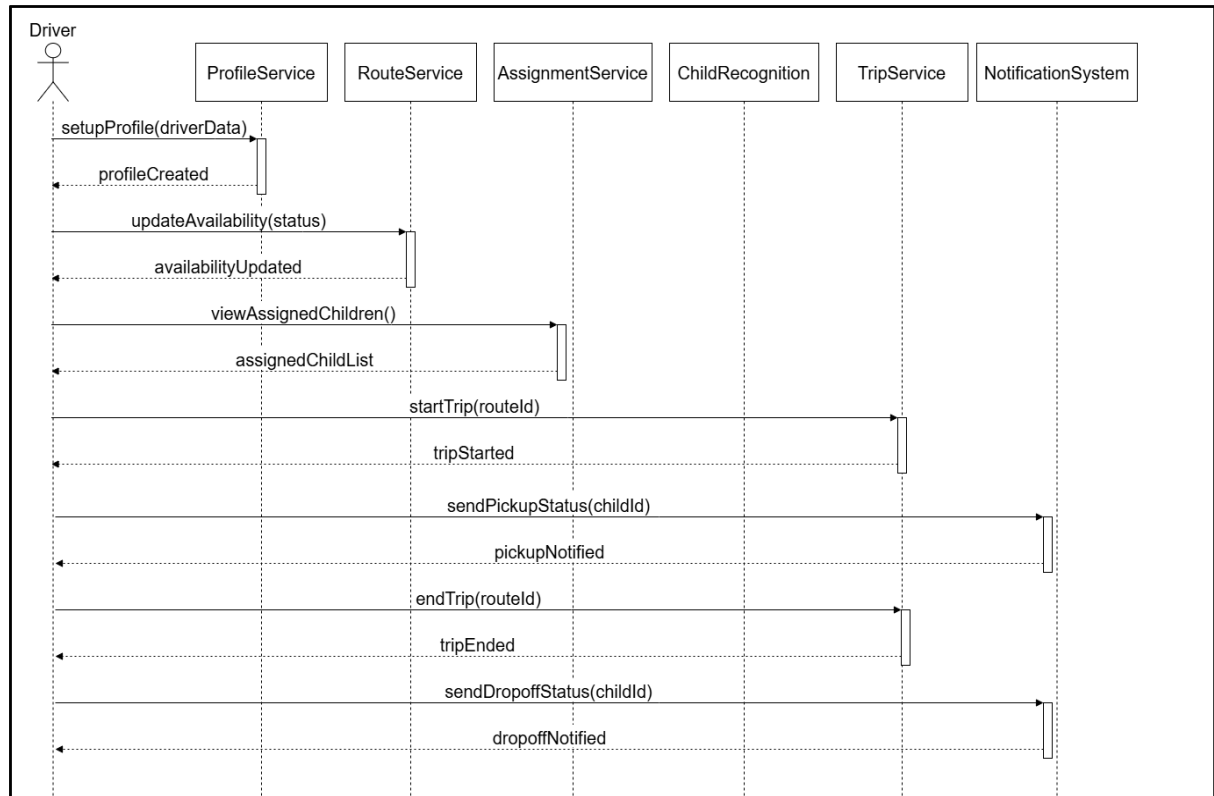


Figure 4.10: Sequence Diagram of Driver

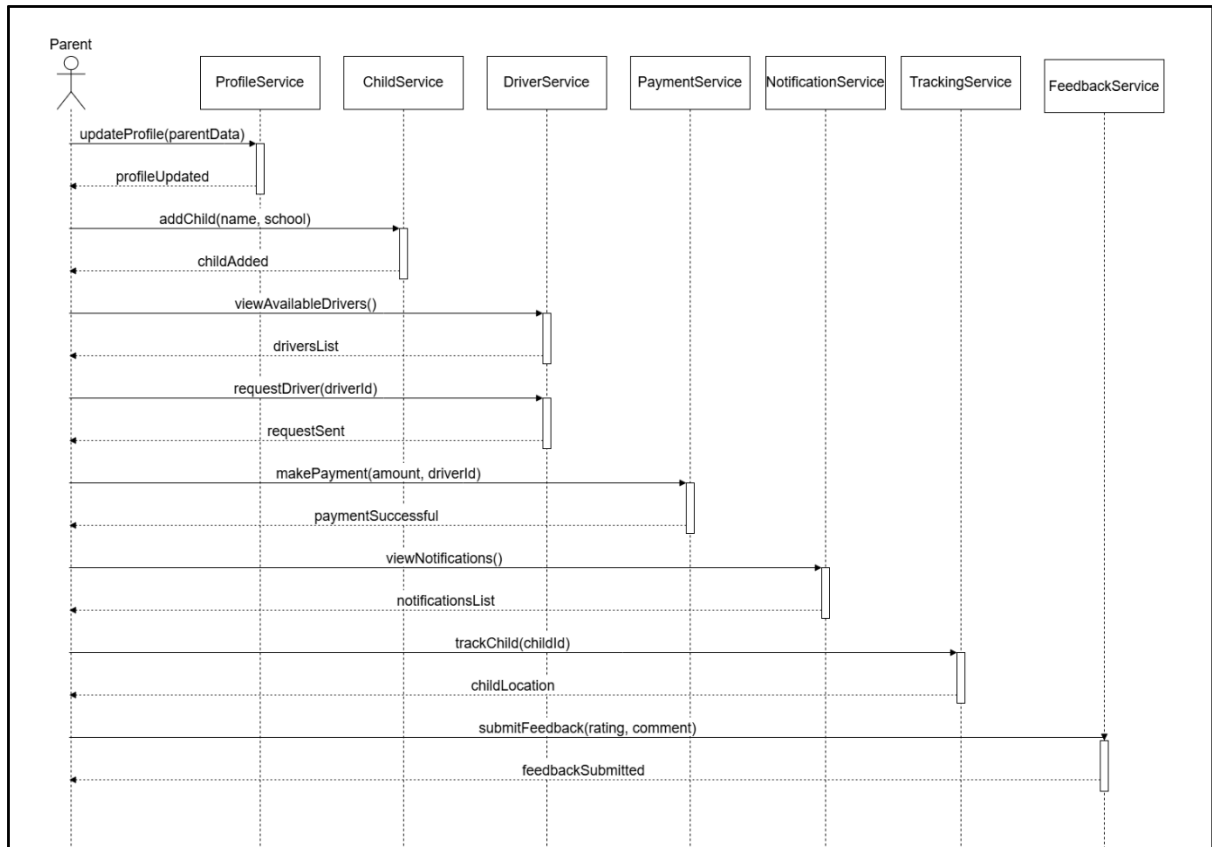


Figure 4.11: Sequence Diagram of Parent

5. Implementation

This chapter provides implementation details of the developed system. The system is a mobile-based School/College Pick-and-Drop Management System developed using Flutter, Firebase, FastAPI, and Stripe. Implementation focus is on system work-flows; algorithmic design; and integration to external services, rather than on source code.

5.1. Algorithm

The system implements a rolebased event-driven architecture where authentication (login, approval of users), ride lifecycle, payment processing, and feedback analysis is carried out by systematic workflows. The key algorithms of the system are explained below.

5.1.1 User Authentication and Role-Based Access Control

1. Authentication process facilitates secure login and role-based navigation.
2. The user interface has input fields to enter login credentials.
3. The input is validated to confirm all necessary fields in the form are entered.
4. The user credentials are verified by firebase authentication which returns a unique user ID (UID).
5. The user profile from firestore using users/{uid}.
6. The user role (Admin, Parent, Driver) is determined from the downloaded user profile.
7. The role-based access controls the next set of rules:
 - Parents will be rendered to the parent dashboard.
 - Admin will be rendered to the Admin Panel.
 - Drivers will have dashboard access only if:
 1. Email is verified
 2. Status (Admin approval) is active
8. In case of any above conditions not being satisfied, the user is rerouted to either pending or restricted access screen.

5.1.2 Driver Registration and Administrative Approval Workflow

This process controls onboarding and verification of driver.

1. Driver fills registration details by filling in personal and vehicle information.
2. Compulsory documents (CNIC, driving license, vehicle documents) are uploaded to firebase storage.
3. The driver profile details are saved in firestore having default status of “inactive”.
4. The admin will check the applications submitted on the admin panel.
5. If the application is verified by the admin, the status of the driver will be updated to “active”.
6. The system re-validates the status of the driver at the time of login and give access to the dashboard only if the status is approved.

5.1.3 Ride Creation and Lifecycle Management

This workflow includes handling the full ride process of assignment to completion.

1. Admin selects students to a verified driver.

2. A ride record is generated in firestore with the initial status (e.g. “ongoing” or “pending”).
3. Driver initiates the ride , and system updates the ride status to “in progress”.
4. Upon completion, driver indicates the “completed” option for ride.
5. Completed rides are released to the parent dashboard for payment processing.

5.1.4 Payment Processing Workflow (Stripe Integration)

The platform is using Stripe for online payment system.

1. A parent chooses a completed ride and proceeds to pay.
2. The flutter application sends request to the backend to create PaymentIntent.
3. The FastAPI backend returns a stripe PaymentIntent and a client secret.
4. The flutter application confirms payment with:
 - PaymentSheet (Android/iOS)
 - PaymentElement (Web)
5. On successful payment:
 - Payment record is saved on Firestore
 - Earnings distribution (E.g. driver/admin split) is calculated and saved
 - Ride status is updated to “paid”
6. Failed or cancelled payment doesn't do any database update.

5.1.5 Sentiment Analysis Workflow (FastAPI + VADER)

This module applies Natural language processing to analyze parent feedback.

1. The parent posts a textual review once the ride is over.
2. The review is submitted to FastAPI/sentiment endpoint.
3. Backend uses VADER sentiment analyzer to process text.
4. The sentiment score is calculated and classified as:
 - Positive
 - Neutral
 - Negative
5. Result is returned to the flutter application.
6. Review and sentiment label stored in firestore for monitoring and analysis.

5.1.6 System Integrity Rule

To guarantee the financial accuracy and reliability of the system:

- A ride is only considered “paid” after successful confirmation from Stripe.
- Earnings are paid only after payment verification.
- Failed or cancelled transactions do not affect the firestore financial records.

5.2. External APIs

It defines the external APIs integrated with the system to support required functionalities.

Table 5.1: Details of APIs used in the project

Name of API	Description of API	Purpose of Usage	Functions / Classes Used
Firestore Authentication	Cloud-based authentication service for user login, registration, and session management.	Provides secure authentication and role-based access control for Admin, Parent, and Driver users.	LoginScreen._login(), SignupScreen._signup(), AuthGate.build(), AuthService.signInWithEmailAndPassword()
Cloud Firestore	Firebase NoSQL real-time database.	Stores user profiles, ride records, payments, reviews, notifications, routes, and other application data.	ParentDashboard, DriverDashboard, AdminDashboard, AuthGate
Firebase Storage	Cloud storage service for files and media.	Stores CNIC images, driving licenses, vehicle documents, profile photos, and other uploaded files.	DriverDashboard.uploadDocuments(), image upload functions
Google Maps SDK	Interactive map rendering and visualization service.	Displays live maps, ride tracking, driver locations,	GoogleMap, DriverLocationScreen, DriverRideMapScreen, ParentDashboard

		routes, and navigation information.	
Google Directions API	Route generation and navigation service.	Generates road routes and route polylines between source and destination locations.	DirectionsService.getRoadRoutePoints()
Google Places API	Place search and location details service.	Provides location autocomplete suggestions and converts place selections into coordinates.	_resolvePlaceIdToLatLng(), place search functions
Geolocator (GPS Services)	Device location and GPS tracking service.	Tracks real-time driver and parent locations, calculates distance, and monitors routes.	Geolocator.getCurrentPosition(), getPositionStream(), distanceBetween()
Stripe API	Secure online payment processing platform.	Processes ride fare payments using PaymentIntent and PaymentSheet mechanisms.	createPaymentIntent(), _payRideWithStripe(), Stripe.instance.initPaymentSheet()

FastAPI REST API	Python-based backend API framework.	Handles sentiment analysis requests and Stripe PaymentIntent creation.	/sentiment, /create-payment-intent, main.py
VADER Sentiment Analysis	Natural Language Processing (NLP) sentiment analysis library.	Analyzes parent reviews and classifies them as positive, neutral, or negative.	SentimentIntensityAnalyzer, polarity_scores(), _classify_vader_with_overrides()

5.3. User Interface

The system user interface is designed to support four different user roles namely Parent, Driver, Admin and Super Admin. Each role has its own interface, so users can only access the functions that are within the scope of their responsibilities. This UI based on roles enhances usability, decreases system complexity, and bolsters access control on the platform. The interface is organised in the form of a dashboard and the user is taken to his/her own dashboard after authentication. Role based modules are provided on the dashboards which help with the core operations of the system like rides management, user management, payment processing, and system monitoring.

This overall design is simple, consistent and responsive. Its consistent and intuitive user experience in mobile and web is enhanced by the use of common UI components like cards, forms, buttons, and status indicators across the system.

5.3.1 Interface Design Principles

The system user interface is designed according to following design principles:

- **Role Based Visibility:** Different user roles can access only the relevant modules and features.
- **Consistency:** Standard UI components are consistently used throughout all screens (cards, forms and status badges).
- **Task Oriented Navigation:** Dashboards show high frequency functions for quick access.
- **System Feedback:** The interface provides immediate feedback with loading indicators, success messages and failure messages.

5.3.2 Parent Interface

The parent interface is used for parents monitoring transport, management of children and processing of payments. It is easy and mobile-friendly to use on a daily basis.

Key features:

- **Dashboard Overview:** Display the status of child transport, open notifications and ride updates.
- **Children Management:** User can add and edit children properties with the school and the route information.
- **Assigned Driver View:** Shows driver details and ride status.
- **Payment Module:** Shows the completed rides with fare information and payment status, a “Pay Now” button.
- **Reviews Module:** Provides parents with feedback and system generated sentiment analysis results.

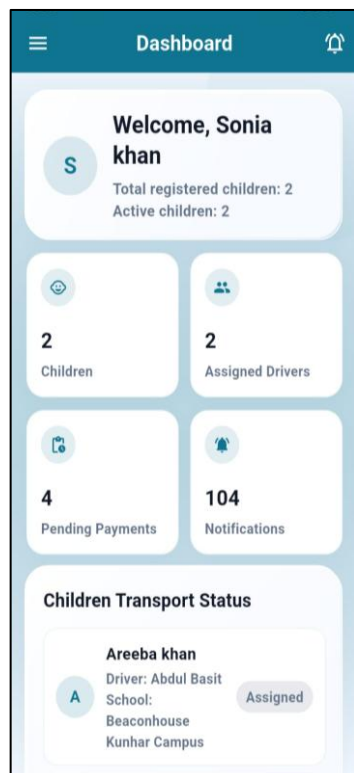


Figure 5.1: Parent Dashboard

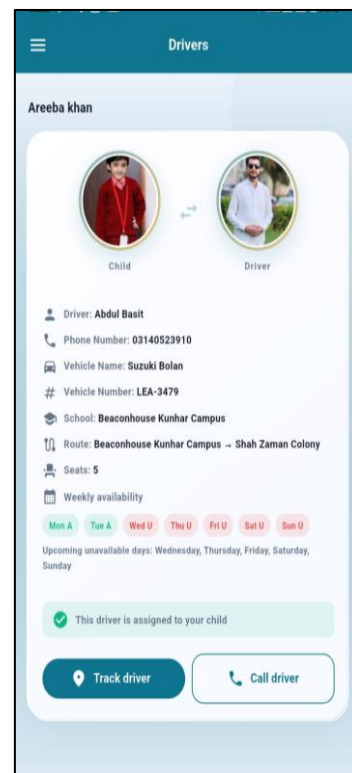


Figure 5.2: Assigned Driver

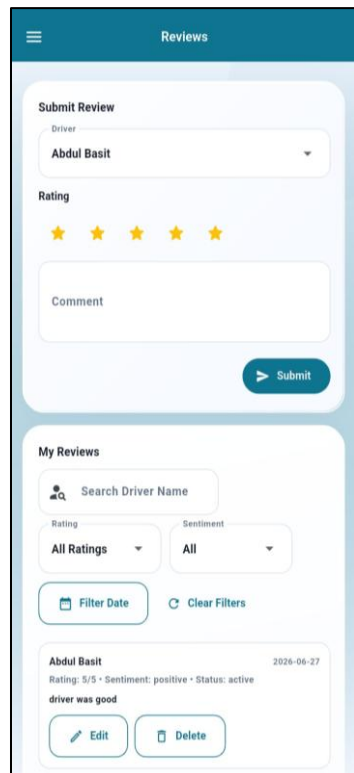


Figure 5.3: Parent Reviews

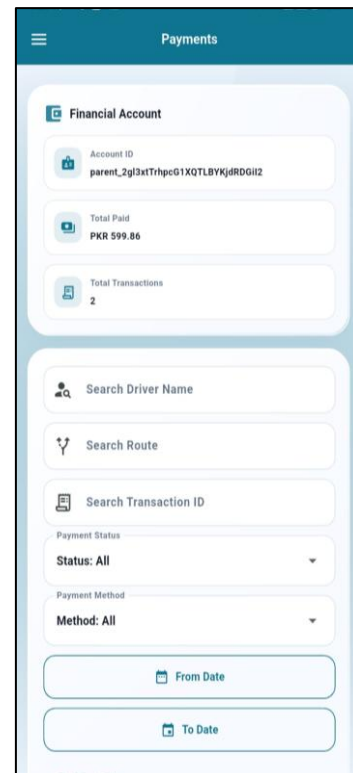


Figure 5.4: Parent Payments

5.3.3 Driver Interface

The driver interface is optimized for operation management and real-time ride execution.

The main components are:

- **Profile and Vehicle Management:** Maintains personal/vehicle information.
- Choose and manage routes and schools assigned.
- **Ride Operations:** Enables drivers to initiate and conclude rides while being updated in real-time.
- **Availability Control:** Allows drivers to indicate availability/availability denied for service.
- **Earnings Dashboard:** Displays ride based earnings and payment highlights. The interface provides intuitive operation flow and state transitions in real time (available, in-progress, completed).

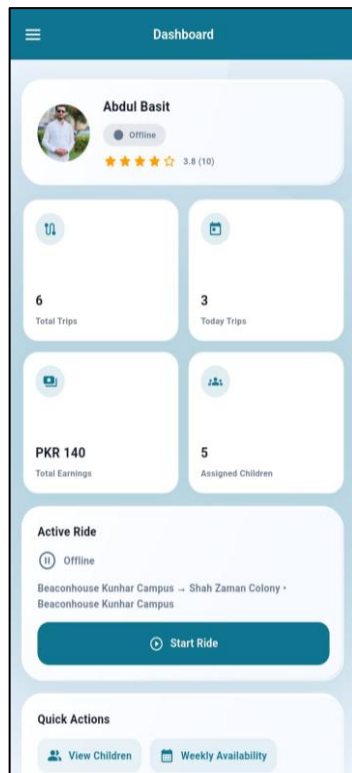


Figure 5.5: Driver Dashboard

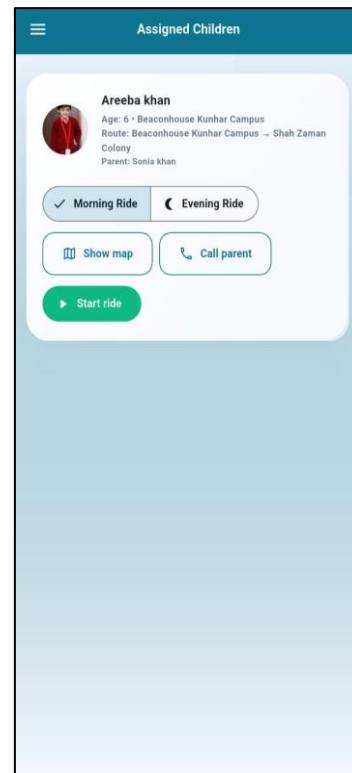


Figure 5.6: Assigned Children

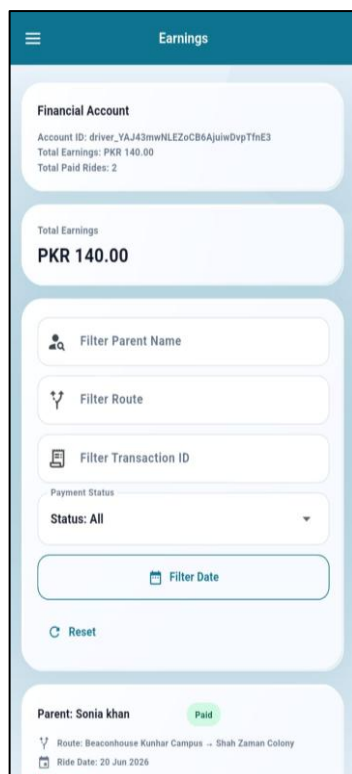


Figure 5.7: Driver Earnings

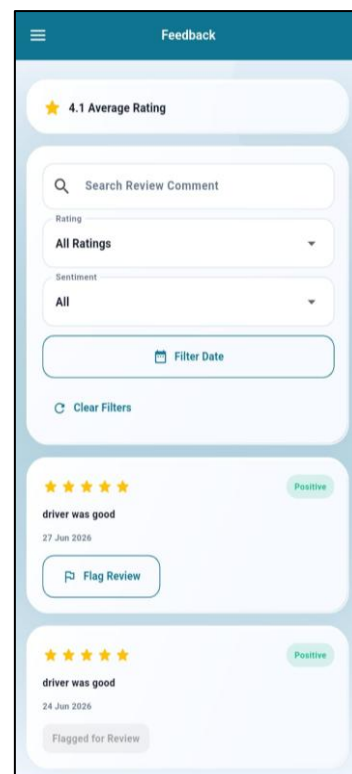


Figure 5.8: Driver Feedback

5.3.4 Admin Interface (Web-Based)

The admin interface is a web based dashboard for administrative control and system monitoring.

Important modules are:

- **Driver Approval Panel:** Approve or decline driver registrations.
- **User Management:** Checks parent and driver accounts and their status of activity.
- **Route and School Management:** Controls routes, schools and fare structures.
- **Request Handling:** Handles requests for transport-related assignments.
- **Reports and Logs:** Shows system activity logs, payment data and feedback summaries.

It is an open platform for managing structured data and making sound decisions.

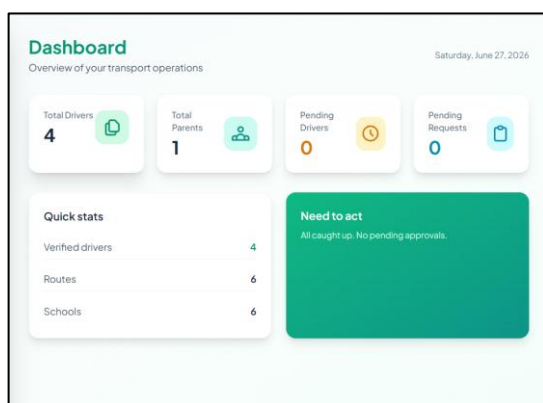


Figure 5.9: Admin Dashboard

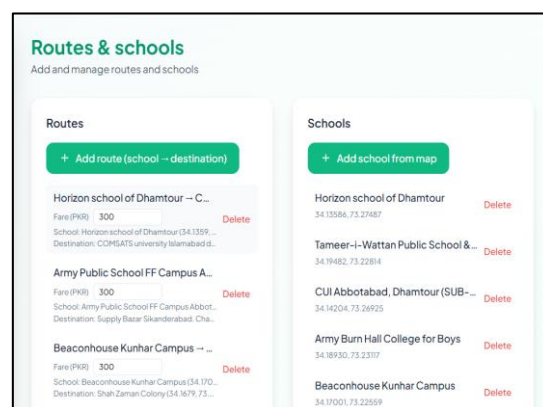


Figure 5.10: Routes and Schools

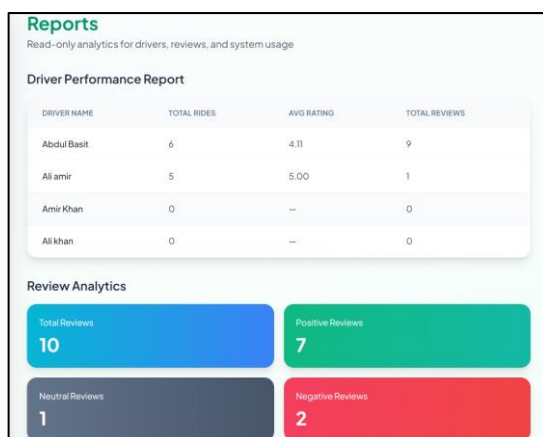
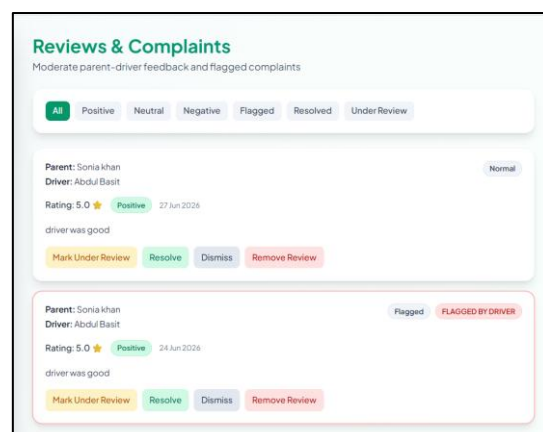


Figure 5.11: Reports



5.12: Reviews and Complaints

5.3.5 Super Admin Interface

Super Admin interface provides system-wide governance and platform-wide monitoring. Main features are:

- **Global Dashboard:** Provides an overview of the whole system activity and the distribution of the Users.
- **Administrative Oversight:** Monitors administrative tasks and implements system policies.
- **Financial Monitoring:** Monitors platform income, payments and distribution of income.
- **Audit Logs:** Keeps track of important activities in the system for transparency and accountability.
- **System Configuration:** Maintains overall and multi-module configurations. This interface is not for day to day use – it is for strategic control and monitoring.

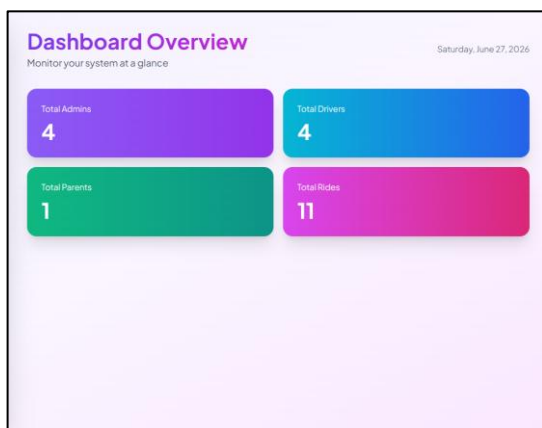


Figure 5.13: Super Admin Dashboard

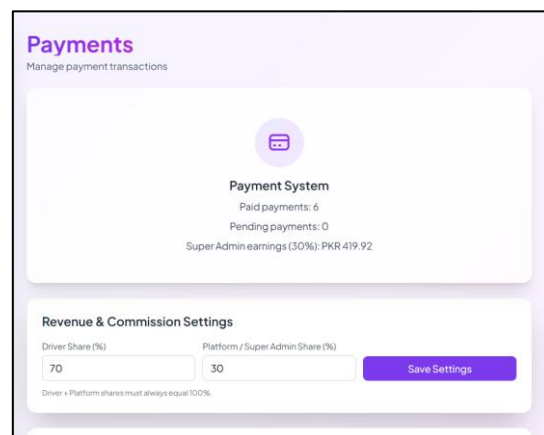


Figure 5.14: Payment Management

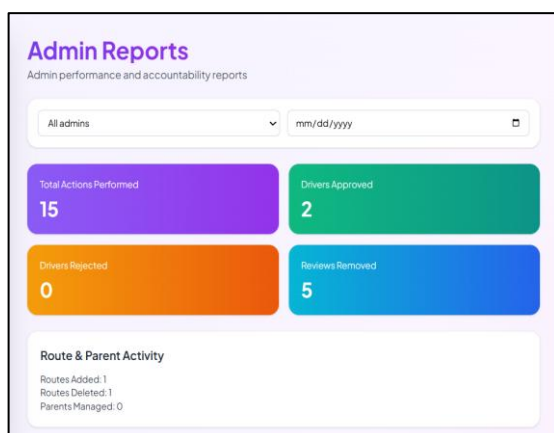


Figure 5.15: Reports

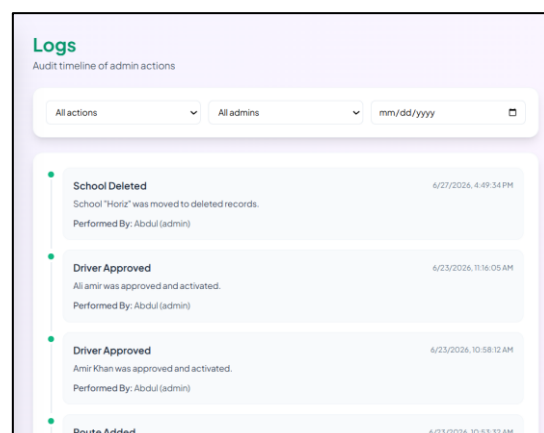


Figure 5.16: Activity Logs

6. Testing and Evaluation

The verification and validation approach taken for the School/College Pick & Drop Management System is discussed in this chapter. The purpose of the testing is to ensure that the app is working correctly, has proper role-based access, financial transactions are secure and are integrated with the Flutter, Firebase, FastApi, and Stripe services.

Manual and automated approaches were used to test to ensure system behavior in the real-world operational conditions.

6.1 Manual Testing

Manual testing carried out to test end-to-end user workflows in a real-world scenario. The goal was to find problems that could not be identified by applying isolated unit testing or inconsistencies in access privileges, as well as errors in system behavior based on the user's role.

The testing process included the following: authentication, driver onboarding, ride lifecycle management, payment processing and review submission with sentiment analysis. All the test cases were checked against the expected system behaviour and were made sure to fulfil the functional requirement.

6.1.1 System Testing

The entire integrated system was subjected to system testing to ensure the overall functionality and interoperability of all system components. The Flutter mobile app, Firebase backend services, FastAPI sentiment analysis service, and Stripe payment gateway.

The main goal of system testing was to make sure that the all modules work properly together and do not cause the data to be inconsistent in the system or cause the workflow to fail.

Table 6.1: System Test Cases

Test Case	Input Data	Expected Result	Actual Result
ST-01: Authentication and Role Routing	Valid credentials for Parent, Driver, Admin, Super Admin	User is redirected to correct dashboard based on role and approval status	Pass
ST-02: Driver Approval Enforcement	Driver account with pending approval	Access denied and pending approval screen displayed	Pass
ST-03: Ride Lifecycle Execution	Driver starts and completes ride	Ride status updates correctly with timestamps	Pass
ST-04: Payment Processing	Completed ride payment via Stripe	Payment succeeds and Firestore updates correctly	Pass
ST-05: Sentiment Analysis Workflow	Parent submits review text	Sentiment classified and stored successfully	Pass

ST-06: Data Consistency Check	Concurrent operations on rides/payments/users	No conflicting or inconsistent states observed	Pass
-------------------------------	---	--	------

6.1.2 Unit Testing

In the system development process, unit testing was done successfully to check and verify working of individual modules and functions. The tested features included authentication, user registration and validation, approval workflow and payment processing.

- **Unit Testing 1: Parent Login**
- **Testing Objective:** To ensure that parents can log in with valid login credentials.

Table 6.2: Parent Login Unit Test Case

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	Verify parent login after clicking the Login button with correct credentials	Email: <u>parent@test.com</u> Password: *****	Parent successfully logs into the Parent Dashboard	Pass

- **Unit Testing 2: Driver Login Approval Verification**
- **Testing Objective:** To make sure only approved and verified drivers are able to access the Driver Dashboard.

Table 6.3: Driver Approval Unit Test Case

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	Verify approved driver login	Approved Driver Account	Driver Dashboard loads successfully	Pass
2	Verify unapproved driver login	Driver Account with pending approval	Access blocked and approval message displayed	Pass

- **Unit Testing 3: Driver Registration Validation**
- **Testing Objective:** To make sure the fields on the driver registration form are validated when registering the driver.

Table 6.4: Driver Registration Unit Test Case

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	Submit registration form with missing required fields	Empty CNIC, License Number, Vehicle Information	Registration rejected and validation message displayed	Pass

2	Submit registration form with invalid CNIC, email, or phone number	Invalid format values	Validation error displayed	Pass
---	--	-----------------------	----------------------------	------

- **Unit Testing 4: Admin Login**
- **Testing Objective:** To make sure that the admin panel can be accessed by the right person with the right credentials.

Table 6.5: Admin Login Unit Test Case

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	Verify admin login after clicking Login button	Admin Email and Password	Admin Dashboard loads successfully	Pass

- **Unit Testing 5: Stripe Payment Processing**
- **Testing Objective:** To get the correct payment processed via Stripe PaymentIntent.

Table 6.6: Payment Processing Unit Test Case

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	Generate Stripe PaymentIntent for a completed ride	Ride ID, Parent ID, Fare Amount	PaymentIntent generated successfully	Pass
2	Simulate payment failure	Backend/API failure scenario	Error displayed and database remains unchanged	Pass
3	Complete payment successfully	Valid payment details	Payment recorded and ride status updated to Paid	Pass

6.1.3 Functional Testing

Functional testing was conducted after each unit was tested to ensure that each part of the system was operating within the stated requirements. The testing was directed towards validating full user workflows and making sure the system performs as expected from the end user point of view.

- **Functional Testing 1: Login with Different User Roles**
- **Objective:** To ensure that the correct dashboard and navigation options are loaded according to the user's role.

Table 6.7: Login Functional Test Case

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	Login as Parent	Valid Parent Email and Password	Parent Dashboard loads with parent navigation menu	Pass
2	Login as Driver	Approved Driver Account	Driver Dashboard loads with driver navigation menu	Pass
3	Login as Admin	Valid Admin Credentials	Admin Dashboard loads with admin navigation menu	Pass

- **Functional Testing 2: Parent Dashboard Operations**
- **Objective:** To ensure that all parent functionalities work correctly.

Table 6.8: Parent Dashboard Functional Test Case

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	Access Parent Dashboard	Parent Account	All parent modules and menus load successfully	Pass
2	View assigned rides and driver details	Existing ride data	Ride information displayed correctly	Pass

- **Functional Testing 3: Driver Dashboard Operations**
- **Objective:** To ensure that all driver-related functionalities operate correctly.

Table 6.9: Driver Dashboard Functional Test Case

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	Access Driver Dashboard	Approved Driver Account	Driver modules load successfully	Pass
2	Perform ride operations	Start Ride and Complete Ride actions	Ride status updates correctly in the system	Pass

- **Functional Testing 4: Ride Management Workflow**
- **Objective:** To ensure that the complete ride lifecycle functions correctly.

Table 6.10: Ride Workflow Functional Test Case

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	Start a ride	Assigned Ride	Ride status changes to “In Progress”	Pass
2	Complete a ride	Active Ride	Ride status changes to “Completed” and payment status remains pending	Pass

- **Functional Testing 5: Payment Processing**
- **Objective:** To ensure that the payment system works correctly using Stripe integration.

Table 6.11: Payment Functional Test Case

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	Complete payment for a ride	Completed Ride with Pending Payment	Stripe payment processed successfully and recorded in the database	Pass
2	Verify payment status update	Successful Payment	Ride payment status updated to “Paid”	Pass

- **Functional Testing 6: Review and Sentiment Analysis**
- **Objective:** To ensure that parent feedback and sentiment analysis functionality operate correctly.

Table 6.12: Review Submission Functional Test Case

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	Submit review for driver	Review Text and Rating	Review stored successfully in the database	Pass
2	Analyze review sentiment	Positive/Negative Review Text	Correct sentiment label generated and stored	Pass

6.1.4 Integration Testing

Integration testing was performed to ensure that different modules of the system communicate correctly with each other and with external services. The testing verified data flow between authentication, ride management, payment processing, sentiment analysis, and approval workflows.

Table 6.13: Integration Test Case

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	Login and dashboard routing integration	Valid Parent, Driver, and Admin credentials	User is authenticated successfully and redirected to the correct role-based dashboard	Pass
2	Ride completion and payment activation	Driver completes a ride	Ride status changes to Completed and payment module becomes available to the parent	Pass
3	Stripe payment and Firestore integration	Successful Stripe payment transaction	Payment record, earnings ledger, and ride payment status are updated correctly in Firestore	Pass
4	Review submission and sentiment analysis integration	Parent submits a review	Review is sent to the FastAPI service, sentiment is analyzed, and result is stored in the database	Pass
5	Admin approval and driver access integration	Admin approves a driver account	Driver approval status updates successfully and driver gains access to the Driver Dashboard	Pass

6.2 Automated Testing

The developed **School/College Pick and Drop Management System** was tested using automated UI testing to verify the functionality, reliability, and correctness of the Super Admin Dashboard. Automation was performed using Selenium WebDriver with Java and TestNG following the Page Object Model (POM) design pattern. The automated tests covered Super Admin login, dashboard navigation, admin management, payment and commission settings, financial transaction filtering, activity logs, deleted record recovery, admin reports, and logout functionality. Explicit waits were used to synchronize the test execution with the application interface and ensure that elements were available before interaction.

6.2.1 Tools Used

Table 6.14 shows the tools used for automated testing.

Table 6.14: Tools Used

Tool Name	Tool Description	Applied on (Related Test Cases / Functional Requirements)	Results
Selenium WebDriver	Web automation framework used to automate browser-based user interactions and	Super Admin login, dashboard navigation, Manage Admins, Payments, Payroll navigation, Logs, Deleted Records, Admin Reports, and Logout.	Core Super Admin UI workflows were executed successfully.

	verify the functionality of the Super Admin Dashboard.		
Java	Programming language used to develop the Selenium automation test scripts and Page Object classes.	Super Admin functional workflows, form interactions, dropdown selections, filtering, navigation, and validation.	Test scripts executed successfully through the Java-based automation framework.
TestNG	Testing framework used to define and execute automated test cases and report test execution results.	Super Admin login and end-to-end dashboard workflow testing.	Test cases were executed and failures were identified during automation execution.
Page Object Model (POM)	Design pattern used to organize web-page elements and actions into separate reusable page classes.	LoginPage, SuperAdminDashboardPage, ManageAdminPage, PaymentsPage, PayrollPage, SuperAdminLogsPage, DeletedRecordsPage, and AdminReportsPage.	Improved test organization, reusability, and maintainability.
Google Chrome	Web browser used as the execution environment for Selenium UI automation.	Complete Super Admin Dashboard workflow and browser-based functional testing.	Application pages and UI interactions were successfully automated in headed mode.

6.2.2 Super Admin Automation Coverage

The automated Super Admin workflow covered the following sequence:

❖ Super Admin Login

❖ Dashboard Verification

❖ Manage Admins

- Open Manage Admins
- Add a new admin using email, password, and city
- Restore a deleted admin

❖ Payments

- Open Payments
- Set Driver Share to **70%**
- Set Super Admin/Platform Share to **30%**
- Save commission settings
- Filter Financial Transaction Ledger using parent name, driver name, transaction ID, payment status, and date

❖ Payroll / Admin Salaries

- Navigation to Payroll/Admin Salaries was automated.
- **Salary payment was removed from the final workflow** because it redirected to the Stripe payment credentials screen.

❖ Logs

- Filter by **All Actions**
- Filter by **Abdul (admin)**
- Filter using the specified date

❖ Deleted Records

- Open Deleted Records
- Restore a deleted review/record

❖ Admin Reports

- Open Admin Reports
- Select the required admin
- Filter/view the report using the specified date
- Scroll through the complete report

❖ Logout

- Click Logout
- Verify that the system returns to the login page

7. Conclusion and Future Work

In this chapter we present final results of School/College Pick-and-Drop Management System and describe future extensions technically feasible for wider deployment and operational maturity.

7.1. Conclusion

The objectives of the Final Year Project have been successful accomplished by creating a complete School/College Pick-and-Drop Management System using Flutter, Firebase, FastAPI, and Stripe. The developed platform is role based in nature with an integrated architecture that allows Parent, Driver, Admin and Super Admin operations within a single workflow of the transport management system.

We are using Firebase Authentication for secure authentication and user session management and Cloud Firestore for managed real-time structured data. Roles are defined for access control and routing so that each type of user can only access the features of the system that are relevant to them, improving the usability, security and governance of the operations.

It also features a GPS tracking system and a notification system, which enables parents and drivers to monitor the status of the ride and provide timely notification. These features enhance transparency, safety and coordination in the transport process.

Operations-wise, the system manages the entire ride life cycle, from ride initiation to active tracking to completion and is kept in status. You can use the payment integration with Stripe, which we will manage to process payments safely and under control, and update the database.

The system is also integrated with a sentiment analysis module using FastAPI and VADER model for sentiment classification of parent feedback. This enables better structuring of the monitoring of service quality and user satisfaction.

The system offers trustworthy and secure role management and functionality in real life applications for the institutional transport coordination in general. The results validate the proposed architecture for typical use cases and the scalability to future needs.

7.2. Future Work

The design of the system is complete within the scope of the project but further enhancements are recommended to increase regional adaptability, scalability and long-term system efficiency.

7.2.1. Specific implementation of JazzCash Payment Gateway.Implementation of JazzCash Payment Gateway.

A possible enhancement to this may be to adopt JazzCash as an alternative or a replacement for Stripe, to better merge the system with the financial infrastructure and payment habits in Pakistan.

7.2.2. Improved Real-Time GPS Tracking.Better Real-Time GPS Tracking.

The accuracy of GPS tracking can be enhanced and predictive ETA generation and route deviations alerts can be included. This will further improve the level of safety monitoring and operational reliability.

7.2.3. Scalable for Multi-City deployment.Scalable for Multi-City deployment.

Data partitioning, Location Based Filtering and Distributed Operational Management allows the system to be extended to support multiple cities and institutions.

7.2.4. Backend Performance Optimization

There is also the potential to further optimize the backend services to enhance the performance of the system in high concurrency situations, particularly for the ride updates, payment transactions and real-time tracking data.

8. References

- [1] A. Ahmed, M. R. Parvez, M. H. Hasan, F. N. Nur, N. N. Moon, A. Karim and e. al., “An intelligent and secured tracking system for monitoring school bus,” in *2019 International Conference on Computer Communication and Informatics (ICCCI)*, Coimbatore, India, 2019.
- [2] S. Desai, R. Suthar, V. Yadav, V. Ankar and V. Gupta, “Smart Bus Fleet Management System Using IoT,” in *Book Title: 2022 Fourth International Conference on Emerging Research in Electronics, Computer Science and Technology (ICERECT)*, conference city, 2022.
- [3] M. Shafiulla, A. L. H. PS, S. M. Naveed, S. Ahmed and U. Kumar, “Revolutionizing Bus Tracking and Arrival Prediction in Real Time for Both Public and Private Sectors, Powered by Firebase,” in *2024 Third International Conference on Distributed Computing and Electrical Circuits and Electronics (ICDCECE)*, 2024.
- [4] D. Hemalatha, B. Nithin and V. Y. Goud, “Browser-Based Real-Time Vehicle Tracking with Simulated V2I Communication Using Firebase Firestore,” in *2026 International Conference on Innovative Computing, Intelligent Communication and Smart Electrical Systems (ICSES)*, 2026.
- [5] J. T. Raj and J. Sankar, “IoT Based Smart School Bus Monitoring and Notification System,” in *2017 IEEE Region 10 Humanitarian Technology Conference (R10-HTC)*, 2017.
- [6] B. Gadade, A. O. Mulani and A. D. Harale, “IoT Based Smart School Bus and Student Monitoring System,” *Naturalista Campano*, vol. 28, no. 1, p. 730–737, 2024.
- [7] E. S. Amer, S. Darwish, N. N. Fahim, M. M. M. M. EL-Qabbary, K. B. Ghaly and A. S. Ramdan, “Vehicle Live Tracking System Based on GPS and GSM,” in *2024 International Telecommunications Conference (ITC-Egypt)*, 2024.
- [8] V. Patil, “Real Time Public Transportation Tracking System,” *Journal of Scientific Research and Technology*, vol. Patil2026, p. 323–337, 2026.
- [9] S. Desai, R. Suthar, V. Yadav, V. Ankar and V. Gupta, “Smart Bus Fleet Management System Using IoT,” in *2022 Fourth International Conference on Emerging Research in Electronics, Computer Science and Technology (ICERECT)*, 2022.
- [10] R. Bandal and A. Oak, “Managing Location Identification and Chain SMS for Smart School Transport System Using IoT,” in *2019 International Conference on Computer Communication and Informatics (ICCCI)*, Coimbatore, India, 2019.

- [11] A. Al-Lawati, S. Al-Jahdhami, A. Al-Belushi, D. Al-Adawi, M. Awadalla and D. Al-Abri, "RFID-Based System for School Children Transportation Safety Enhancement," in *2015 IEEE 8th GCC Conference & Exhibition*, 2015.
- [12] S. Hafeez, S. S. Alotaibi, A. Alazeb, N. Al Mudawi and W. Kim, "Multi-sensor-based action monitoring and recognition via hybrid descriptors and logistic regression," *IEEE Access*, vol. 11, p. 48145–48157, 2023.
- [13] M. W. Raad, M. Deriche and T. Sheltami, "An IoT-Based School Bus and Vehicle Tracking System Using RFID Technology and Mobile Data Networks," *Arabian Journal for Science and Engineering*, vol. 46, no. 4, p. 3087–3097, 2021.
- [14] H. Gull, D. Aljohar, R. Alutaibi, D. Alqahtani, M. Alarfaj and R. Alqahtani, "Smart School Bus Tracking: Requirements and Design of an IoT Based School Bus Tracking System," in *2021 5th International Conference on Trends in Electronics and Informatics (ICOEI)*, Tirunelveli, India, 2021.
- [15] R. C. isha, A. Jyothindranath and L. S. Kumary, "IoT Based School Bus Tracking and Arrival Time Prediction," in *2017 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*, Udupi, India, 2017.

thesis (4)		
ORIGINALITY REPORT		
13%		
SIMILARITY INDEX		
PRIMARY SOURCES		
1	www.coursehero.com Internet	464 words — 3%
2	dspace.daffodilvarsity.edu.bd:8080 Internet	216 words — 1%
3	vehari.comsats.edu.pk Internet	197 words — 1%
4	jcbi.org Internet	120 words — 1%
5	eprints.utar.edu.my Internet	112 words — 1%
6	Fortes, Shirley Fermino Rodrigues. "Implementation of an APP for Supporting Best Practices in Tuberculosis Treatment.", Universidade do Porto (Portugal) ProQuest	82 words — < 1%
7	northandsouthdigital.com Internet	67 words — < 1%
8	hdl.handle.net Internet	62 words — < 1%
9	www.scribd.com Internet	51 words — < 1%

10	Ton Duc Thang University Publications	44 words — < 1%
11	R. V. Viswanathan, S Sujatha, S. Ananthi, V.C. Bharathi, V. Sivasakthi, T. Deepa. "AI-Driven Campus Transit: Intelligent Real-Time Bus Tracking and Grievance Management System", 2025 Third International Conference on Emerging Applications of Material Science and Technology (ICEAMST), 2025 Crossref	41 words — < 1%
12	iecsience.org Internet	41 words — < 1%
13	ijrpr.com Internet	39 words — < 1%
14	cookmyproject.com Internet	37 words — < 1%
15	jst.tnu.edu.vn Internet	37 words — < 1%
16	ejournal.sisfokomtek.org Internet	35 words — < 1%
17	Tarneem M. Hamadto, Zakaria A. Adam, M.H. Elsayed. "An Android Application of School Bus Tracker Based on RFID Technology", 2020 International Conference on Computer, Control, Electrical, and Electronics Engineering (ICCEEE), 2021 Crossref	34 words — < 1%
18	Barath E, Hirthicik J S Suriyah, Devanathan B, Lakshmanan S A. "IoT Enabled Real-Time Vehicle Tracking and Alert System for Educational Transport Service", 2025 5th International Conference on Expert Clouds and Applications (ICOECA), 2025	33 words — < 1%

Crossref		
19	www.se.rit.edu Internet	33 words — < 1 %
20	inass.org Internet	32 words — < 1 %
21	dr.ur.ac.rw Internet	30 words — < 1 %
22	etd.aau.edu.et Internet	28 words — < 1 %
23	tudr.thapar.edu Internet	28 words — < 1 %
24	irjet.net Internet	27 words — < 1 %
25	thesai.org Internet	27 words — < 1 %
26	www.ijirset.com Internet	26 words — < 1 %
27	scholarcommons.scu.edu Internet	22 words — < 1 %
28	spectrum.library.concordia.ca Internet	20 words — < 1 %
29	irigs.iiu.edu.pk:64447 Internet	19 words — < 1 %
30	R. Adhi Lakshmi, V. Kaviyashri, M. Lakshmi Sri, G. A. Pradiba, R. Rubika. "Leveraging LSTM for Accurate ETA Classification and Delay Prediction in College Bus	18 words — < 1 %

thesis (4).docx

Quick Submit

Quick Submit

Turnitin

Document Details

Submission ID

trn:oid::1:5052725201

83 Pages

Submission Date

Jul 5, 2026, 6:11 AM

15,854 Words

Download Date

Jul 5, 2026, 6:11 AM

92,278 Characters

File Name

thesis (4).docx

File Size

3.92 MB

Page 2 of 83 - AI Writing OverviewSubmission IDtrn:oid::1:5052725201

*% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

 **0 AI-generated only 0%**
Likely AI-generated text from a large-language model.

 **0 AI-generated text that was AI-paraphrased 0%**
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.